SUNWAY UNIVERSITY | Faculty of ENGINEERING & TECHNOLOGY

CAPSTONE PROJECT 2

Final Project Report

ZY-Path: A GUI-Based AI Histopathology Image Classification Platform

by

LI ZI YAO

24018160

Bachelor of Software Engineering (Hons)

Supervisor: Dr Wan Siti Halimatul Munirah Wan Yahya @ Wan Ahmad

Semester: [TO CONFIRM]

Date: [TO CONFIRM]

School of Computing and Artificial Intelligence
Faculty of Engineering and Technology
Sunway University

Acknowledgements

I would like to thank my supervisor, Dr Wan Siti Halimatul Munirah Wan Yahya @ Wan Ahmad, for her guidance throughout this capstone project, for providing the reference classification training code, and for providing the segmentation model and inference notebook. I conducted the classification training and evaluation and developed the desktop application, including model integration, result inspection, history, export and the associated interaction workflows.

I also acknowledge the researchers and maintainers whose open-source work makes this project possible, particularly the PyTorch, torchvision, Qt/PyQt, Pillow, NumPy, Matplotlib, SQLite and fpdf2 communities. Their software and documentation provided the foundation for implementing and testing ZY-Path. The published digital-pathology and usability research discussed in this report informed the project's scope and evaluation design.

Abstract

ZY-Path is a local PyQt5 desktop application for model-aware histopathology classification, combining Single image and Tiled analysis modes, spatial inspection, SQLite history, recorded rerun and export, with a separate supervisor-provided segmentation integration. On 7,180 CRC-VAL-HE-7K patches, the classifier achieved 90.68% accuracy and 88.59% macro-F1 through both the recorded OpenCV/Albumentations pipeline and the application's default Pillow Single image core, with agreement on all predicted labels. The complete native Windows automated suite passed all 370 tests; UAT, segmentation accuracy, systematic performance measurement and clean-machine deployment remain to be evaluated.

Keywords: digital pathology; desktop application; PyQt5; PyTorch; tiled classification; semantic segmentation; reproducible workflow.

Table of Contents

Acknowledgements.....i
Abstract.....ii
List of Figures.....vii
List of Tables.....ix
Chapter 1: Introduction.....1
1.1 Background and Motivation.....1
1.2 Problem Formulation.....2
1.2.1 Problem Statement.....3
1.3 Hypothesis.....4
1.4 Project Objectives.....4
1.5 Scope of Work.....5
1.6 Research Significance.....7
Chapter 2: Literature Review.....8
2.1 Introduction.....8
2.2 Thematic Analysis of Related Research.....8
2.2.1 Digital histopathology and the unit of analysis.....8
2.2.2 Classification architectures and pretrained deployment.....9
2.2.3 Semantic segmentation and multiscale context.....9
2.2.4 Tiled classification, coverage and confidence.....10
2.2.5 No-code interaction and transparent constraints.....11
2.2.6 Persistence and reproducible workflows.....11
2.2.7 Human-centred design and evaluation.....11
2.3 Review of Existing Solutions and Platforms.....12
2.3.1 QuPath.....12
2.3.2 Slideflow and Slideflow Studio.....12
2.3.3 WSInfer and QuPath integration.....13
2.3.4 Cytomine.....13
2.3.5 Commercial environments.....13
2.4 Comparative Evaluation of Existing Systems.....14
2.5 Identified Research Gaps and Project Positioning.....15
2.6 Technological Infrastructure and Implementation Considerations.....15
2.7 Synthesis of Findings and Project Implications.....16
2.8 Summary.....17
Chapter 3: Methodology.....18
3.1 Overview of System Methodology.....18
3.2 Functional and Non-Functional Requirements.....20
3.2.1 Functional Requirements.....20
3.2.2 Non-Functional Requirements.....21

3.2.3 Ethical Considerations....22
3.2.4 Accessibility Compliance....23
3.3 Methodologies, Tools, and Techniques....23
3.3.1 Model representations and executable validation....23
3.3.2 Classification training and model configuration....24
3.3.3 Classification preprocessing and resize provenance....25
3.3.4 Explicit classification modes and crop geometry....25
3.3.5 Class probabilities, maps and estimated coverage....26
3.3.6 Segmentation pathway and label restoration....27
3.3.7 Background execution and cancellation....29
3.3.8 History schema, artifacts and rerun isolation....29
3.3.9 Export and local model defaults....31
3.4 User Interface (UI) and User Experience (UX) Design....31
3.4.1 Principles and information hierarchy....31
3.4.2 Spatial results and progressive disclosure....32
3.4.3 Control states, settings and help....33
3.4.4 Responsive windows and platform limits....33
3.5 Unified Modeling Language (UML) Diagrams....34
3.5.1 Use Case Diagram....34
3.5.2 Sequence Diagram....35
3.5.3 Class Diagram....36
3.5.4 Activity Diagram....38
3.6 Testing Methodology....39
3.6.1 Development Testing....39
3.6.2 Module Testing and Fallback Handling....39
3.6.3 Manual System Testing....40
3.6.4 User Acceptance Testing (UAT)....40
3.6.5 Testing Summary....41
3.6.6 Requirements Traceability Matrix....42
3.7 Risk Management and Mitigation Strategies....44
3.8 Summary....45
Chapter 4: Results and Discussion....47
4.1 Introduction....47
4.2 System Implementation Results....47
4.2.1 Implemented workflow and scope....47
4.2.2 Classification results and spatial inspection....48
4.2.3 Segmentation results....49
4.2.4 History, rerun and export....49
4.2.5 Settings, cancellation and interface polish....50
4.2.6 Classification training and test results....52
4.3 Functional Testing Results....57
4.3.1 Recorded automated results....57

4.3.2 Challenges, resolutions and evidence....59
4.3.3 Windows startup and window-state observations....61
4.3.4 Runtime and deployment verification status....62
4.4 User Acceptance Testing Results....62
4.4.1 Participant Demographics....62
4.4.2 Quantitative Results....63
4.4.3 Feature Usage and Ratings....64
4.4.4 System Performance and Acceptability....64
4.4.5 Qualitative Feedback Analysis....65
4.5 Discussion....65
4.5.1 Achievement of Research Objectives....65
4.5.2 Comparison with Existing Solutions....67
4.5.3 Limitations and Constraints....67
4.5.4 Implications for Future Development....68
4.6 Summary....69
Chapter 5: Conclusion and Future Work....70
5.1 Conclusion....70
5.2 Future Work and Recommendations....71
5.3 Research Contributions....71
5.4 Summary....72
References....73
Appendices....76
Appendix A: Project Timeline and Planning....76
A.1 Proposal baseline....76
A.2 Implementation and outstanding milestones....81
A.3 Completion sequence....82
Appendix B: User Acceptance Testing Details....83
B.1 Participant information and consent briefing....83
B.2 Background questions....83
B.3 Standard task script and observation rules....83
B.4 System Usability Scale questionnaire....85
B.5 Feature-specific ratings and open questions....85
B.6 Scoring, missing data and reporting....85
Appendix C: Implementation and Evidence Index....86
C.1 Source baseline....86
C.2 Evidence interpretation....88
Appendix D: Manual Windows Acceptance and Screenshot Checklist....88
D.1 Native acceptance record....88
D.2 Required figure replacements....89
Appendix E: Independent Model Evaluation Protocol....89
E.1 Classification....89
E.2 Segmentation....90

E.3 Runtime and deployment.....90

List of Figures

Figure 1.1. Representative H&E colorectal tissue image (evidence pending; replace this reserved area with the actual authorised capture).....1

Figure 2.1. Three output meanings: whole-image classification, source-tile estimates and pixel segmentation. Author-created conceptual diagram; it contains no model accuracy results.....10

Figure 3.1. Implemented logical architecture and artifact flow. Source: author, based on the project's Python modules.....19

Figure 3.2. Annotated final workflow wireframe. This author-created design diagram summarises implemented regions and is not a Windows runtime screenshot.....32

Figure 3.3. Use case diagram for the implemented local analysis workflow. File/model trust and clinical interpretation remain external responsibilities. Source: author, based on the implemented application.....35

Figure 3.4. Sequence diagram for a run and its persistence boundary. Cancellation is acknowledged after a safe checkpoint; artifact-save failures must not be represented as successful saved records. Source: author, based on the implemented application.....36

Figure 3.5. Implementation-level class diagram. Relationships represent principal ownership or dependency, not a generated inventory of every method. Source: author, based on the implemented application.....37

Figure 3.6. Activity diagram from parameter selection to completed persistence, with safe cancellation and error exits. Source: author, based on the implemented application.....38

Figure 4.1. Final Windows application overview (evidence pending; replace this reserved area with the actual authorised capture).....47

Figure 4.2. Tiled classification interface with source/result comparison and revised result toggles. Source: author's Linux/offscreen-Qt development capture; synthetic fixture, not a model result.48

Figure 4.3. Real-weight single-image and tiled classification evidence (evidence pending; replace this reserved area with the actual authorised capture).....49

Figure 4.4. Windows segmentation result and aligned comparison (evidence pending; replace this reserved area with the actual authorised capture).....49

Figure 4.5. History and recorded-parameter rerun (evidence pending; replace this reserved area with the actual authorised capture).....50

Figure 4.6. Exported report and companion artifacts (evidence pending; replace this reserved area with the actual authorised capture).....50

Figure 4.7. Implemented Guide & Support page with persistent Contents navigation and numbered guidance. Source: author's Linux/offscreen-Qt development capture; this is interface evidence.....51

Figure 4.8. English history-calendar popup with a simulated Chinese default locale. Source: author's Linux/offscreen-Qt development capture; the selected date is test input.....51

Figure 4.9. Cancellation and per-task model defaults on Windows (evidence pending; replace this reserved area with the actual authorised capture).....52

Figure 4.10. Classification training and validation over 30 recorded epochs, with the selected epoch 26 marked. Source: author's plot of experiments/crc-dn21-seed42-10ep/metrics.csv.....53

Figure 4.11. Nine-class confusion matrix for CRC-VAL-HE-7K. Source: the classification experiment's saved matrix; the Windows default-Pillow predictions independently reproduce every count. Rows are true labels and columns predicted labels.....55

Figure 4.12. Completed Windows pytest report, 19 October 2026. Source: the recorded full-suite log; selected text is reproduced with omissions marked.....59

List of Tables

Table 2.1. Scope-based comparison of pathology platforms....14
Table 3.1. Functional requirements and acceptance conditions....20
Table 3.2. Non-functional requirements and verification boundaries....21
Table 3.3. Classification class order and fixed display palette v1....27
Table 3.4. Built-in segmentation labels and display colours....28
Table 3.5. Implemented persistent data structures....29
Table 3.6. Requirements traceability matrix....42
Table 3.7. Risk register and residual mitigations....44
Table 4.1. Classification evaluation summary....53
Table 4.2. Class-wise classification performance on 7,180 test images....54
Table 4.3. Automated verification records by environment and evaluated source state....57
Table 4.4. Windows test host and actual CPU runtime on 19 October 2026....58
Table 4.5. Implementation challenges and verified resolutions....59
Table 4.6. UAT participant profile — awaiting actual responses....62
Table 4.7. UAT quantitative summary — awaiting valid data....63
Table 4.8. Feature ratings — awaiting actual use and responses....64
Table 4.9. Objective achievement and outstanding evidence....65
Table A.1. Original CP1 planning baseline, retained from the submitted proposal....76
Table A.2. Original CP2 planning baseline, retained from the submitted proposal....78
Table A.3. Evidence-based milestone status....81
Table B.1. UAT tasks and success criteria....83
Table C.1. Principal evidence sources....86
Table D.1. Final Windows acceptance cases....88

Chapter 1: Introduction

1.1 Background and Motivation

Histopathology plays a key role in modern medical care by offering the examination of tissue samples at the microscopic level to aid in diagnosing disease, in predicting disease outcome and in planning therapy. New methods to analyze histopathological morphology have been focused on reducing the manual work of pathologists, which is traditionally the main means for analyzing histopathological features and suffers from intra- and inter-observer variability. In parallel with the fast development of digital pathology and artificial intelligence, medical images have been digitized and analysis/computation on images has been widely applied to enhance productivity and aid clinical decision-making.

Histopathology is a reference method for diagnosing many diseases, including malignancies. In haematoxylin and eosin (H&E) images, the staining of nuclei and cytoplasmic structures provides morphological information that trained professionals interpret in context. Digital imaging makes these observations available for storage, teaching and computational analysis. A model's output, however, is an inference from image appearance and must not be equated with a verified clinical diagnosis (Litjens et al., 2017; Kather et al., 2019).

EVIDENCE TO INSERT — Representative H&E colorectal tissue image

Insert an authorised, de-identified image used in this project, with dataset/sample identifier, source credit and known scale if available. This replaces the proposal's illustrative tissue image only if its provenance cannot be confirmed. Do not label a generated test pattern as histology.

Figure 1.1. Representative H&E colorectal tissue image (evidence pending; replace this reserved area with the actual authorised capture).

Convolutional neural networks support classification and segmentation through learned visual representations. DenseNet provides feature reuse through dense connectivity, while encoder–decoder and multiscale architectures support spatial prediction (Huang et al., 2017; Ronneberger et al., 2015; Xiao et al., 2018). Their practical use also depends on apparently small implementation choices: the channel order, numerical range, interpolation method, class order and image scale must agree with the model's requirements. A visually convincing result does not by itself demonstrate that these choices are correct.

Existing pathology platforms offer graphical inspection and analysis: QuPath provides an extensible image-analysis environment, Slideflow Studio supports whole-slide visualisation, and WSInfer integrates reusable pretrained models with QuPath (Bankhead et al., 2017; Dolezal et al., 2024; Kaczmarzyk et al., 2024). ZY-Path addresses a focused workflow within this landscape by connecting supported local model loading, distinct analysis modes, spatial inspection and persistent inference records in one desktop application.

The software is aimed at students, educators and biomedical researchers who wish to inspect model behaviour without writing inference code. It separates two questions that are often confused: whether an application runs the intended calculation, and whether the resulting predictions are accurate for a clinical or research population. The former can be investigated with controlled software tests. The latter requires appropriately labelled independent data, known sampling conditions and a valid evaluation design.

1.2 Problem Formulation

The application-level problem is to turn a compatible trained model and a raster pathology image into a traceable analysis result through a graphical workflow. An analysis can be represented as R = f(I, M, T, P, G), where I is the source image, M the model, T the selected task and classification mode, P the preprocessing contract, and G the spatial sampling geometry. Results should retain enough of these inputs to explain what was run and to support a subsequent rerun. Display choices, such as overlay opacity, should change presentation rather than the underlying prediction or coverage calculation.

This formulation extends the CP1 image-level workflow to explicit tiled classification. Whole-image classification produces one probability vector for the complete resized image. Tiled classification produces one vector per crop and a spatial class map. Segmentation produces a pixel-label map through a separate model pathway. Treating these outputs as interchangeable would misrepresent their evidence, particularly when a tile-level coverage estimate is mistaken for a pixel-accurate boundary.

The original research questions remain relevant to this implemented scope:

RQ1: Is it feasible to develop a no-code desktop software that enables direct loading of PyTorch models, execution of classification and segmentation tasks and simplifies the application and adoption of pretrained deep learning models in histopathology image analysis without programming requirement?

RQ2: Is it possible to present image classification and segmentation results, including associated confidence information and pixel-wise predictions, in a manner that is both effective and intuitive via a graphical user interface (GUI) to allow for rapid interpretation by users who are not professionally trained?

RQ3: Can the unified integration of inference history management, record retrieval (e.g., search and filtering), and one-click re-execution of prediction for records improve the usability, workflow effectiveness, and scientific reproducibility of the AI-enabled histopathology analysis system?

In this project, direct loading refers to trusted model representations with supported input/output and metadata contracts. Recorded rerun restores a previous analysis configuration under stable files and dependencies. These operational definitions guide the requirements and evaluation; their boundaries are discussed in Section 4.5.3.

1.2.1 Problem Statement

Three connected problems define the project. First, deployment decisions are difficult to inspect when model compatibility, preprocessing and task selection are spread across scripts or tools. ZY-Path needs to expose a simple workflow while rejecting unsupported combinations, retaining the actual model/task information and making consequential parameters visible.

Second, interpretation requires spatial and statistical clarity. A source image, a class-colour map, an overlay and a probability chart answer different questions. A user should be able to compare corresponding coordinates, see the class legend, inspect tile confidence and distinguish estimated tile coverage from segmentation pixel counts. Large images and narrower windows must remain usable without losing essential controls.

Third, completed analyses need continuity. Users should be able to retrieve a record, inspect its saved result, export a report and rerun its recorded model, mode and

parameters. Keeping these settings separate from current preferences makes the previous analysis intelligible and reduces manual reconstruction of its configuration.

1.3 Hypothesis

The project adopts three evaluation propositions corresponding to the original research questions. H1 is that a constrained, model-aware desktop workflow can execute classification and segmentation without users writing inference code. H2 is that explicit visual and statistical distinctions between single-image classification, tile estimates and segmentation will support usable result inspection. H3 is that restoring recorded inputs and parameters will support continuity and reduce the need to reconstruct a previous analysis manually.

H1 can be examined through executable requirements, compatible/incompatible model tests and end-to-end workflow checks. H2 requires more than correct rendering: it also requires task-based user evaluation of interpretation and perceived usability. H3 has a technical component—correct restoration and isolation from current defaults—and a user component concerning successful retrieval and rerun. This report distinguishes the available technical evidence from these untested user outcomes.

For the planned pilot, SUS provides a descriptive measure of perceived usability; its interpretation will use the actual sample, tasks and score distribution (Brooke, 1996; Bangor et al., 2009). Hypothesis assessment combines the technical evidence with the corresponding user evidence rather than treating software tests as responses to a usability survey.

1.4 Project Objectives

The seven CP1 objectives are retained in substance, with the supported-model and evidence limits made explicit:

1. O1: Develop a local, no-code desktop workflow for histopathology image analysis without a running application server.
2. O2: Allow trusted, compatible pretrained PyTorch models to be selected through a graphical file browser and validated before inference.
3. O3: Support classification and segmentation with model-appropriate preprocessing, output parsing and explicit spatial sampling where applicable.

4. O4: Present class labels, probabilities, overlays, independent class legends and source/result comparisons in a readable interface.
5. O5: Export analysis presentations and available metadata as PNG images and structured PDF reports.
6. O6: Persist completed analysis records and associated artifacts locally with their actual task, model and analysis parameters.
7. O7: Support history search, date filtering, record restoration and rerun using the recorded settings.

To support O2 and O3 with model evidence, I also trained and evaluated the classifier using the reference code provided by my supervisor. This supporting research activity supplies a documented checkpoint and patch-level evaluation; it is separate from the inference-only GUI and introduces no new neural architecture.

Tiled analysis, cancellation, task-specific model defaults and responsive comparison views refine these objectives. O1 is implemented through local source execution without an application server; producing and validating a standalone executable is a further deployment objective addressed in Sections 4.3.4 and 5.2.

1.5 Scope of Work

The implemented scope comprises standard raster-image loading, local CPU inference, model validation, two explicit classification modes, compatible segmentation, result inspection, local history, cancellation and export. PNG, JPEG, TIFF and BMP are offered through the image-selection workflow. The pipeline opens the first image frame and converts it to RGB; it does not traverse a multi-resolution slide pyramid.

Single image classifies the complete selected image regardless of its dimensions. Tiled analysis uses the chosen source tile width and height even when the image fits in one tile. Consequently, a small image is not silently assigned a different inference pathway merely because a global large-image switch is enabled. Both modes use the recorded classifier contract.

The built-in classifier uses the ordered classes ADI, BACK, DEB, LYM, MUC, MUS, NORM, STR and TUM. Segmentation has a separate nine-label vocabulary and palette. Classification source tiles begin at an editable 512 × 512 pixels, with training-

scale equivalence explicitly unconfirmed; the consequences of source scale and edge resizing are discussed in Section 4.5.3.

The following CP1 exclusions remain applicable:

AI model training and optimization

The desktop application is inference-only. Classification training was carried out separately using the reference code provided by my supervisor; the resulting model was then integrated into ZY-Path. Training loops and hyperparameter tuning are not exposed through the GUI.

Dataset creation and annotation

No tools are included for labelling images, creating ground-truth masks, or organizing training datasets; the platform is inference-only.

Cloud deployment and web-based services

The system is a native desktop application, it does not offer a web interface, REST API, or any cloud-hosted functionality.

Multi-user collaboration and annotation

Designed for single-user use on a local machine, it does not support shared workspaces, simultaneous editing, or team-based annotation workflows.

Clinical diagnosis and medical certification

The software is intended solely for educational and research purposes; it must not be used for making clinical decisions or issuing medical certifications.

Two exclusions require clarification. Large raster images can now be divided into tiles and reconstructed as a class map; therefore, tile extraction and reconstruction are no longer excluded. Native gigapixel whole-slide reading, pyramid navigation, scanner-specific metadata handling and slide-level diagnostic validation remain outside scope. No Grad-CAM or other explanation method is implemented; a class overlay visualises model output and is not a causal explanation of the model.

Local execution requires the selected model files and, for UPerNet reconstruction, its verified configuration in the local Hugging Face cache. Windows source execution and real-classifier validation are documented in Section 4.3.3; offline package verification is addressed separately in Section 4.3.4.

1.6 Research Significance

This work advances the democratization of AI for digital pathology by addressing the challenges and technical complexities related to model deployment and image analysis.

For education, the interface gives students and researchers a way to explore supported model outputs without writing inference code. Visible modes, class legends, source comparisons and retrievable records connect technical settings to the interpretation of a result.

The engineering contribution is a constrained integration: explicit model contracts, distinct spatial tasks, linked inspection and a record-oriented workflow are combined in a local application. The implementation demonstrates how GUI decisions—such as preserving a record's mode or separating tile coverage from global confidence—can prevent misleading interpretations of technically valid numerical outputs.

SDG 3: Good Health and Well-Being

The project contributes to the advancement of healthcare technologies by supporting AI-assisted histopathology image analysis. While not intended for clinical diagnosis, the system demonstrates how AI can assist medical image interpretation and facilitate future innovations in healthcare.

SDG 4: Quality Education

The project promotes accessible learning opportunities by enabling students and researchers to utilize AI technologies without extensive programming expertise. The no-code approach supports digital literacy and encourages broader participation in AI-assisted medical image analysis.

SDG 3 and SDG 4 provide the wider motivation for the project. The present evaluation concerns engineering behaviour and classification performance; the planned UAT will examine the educational/research workflow from the intended users’ perspective.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the domain concepts, existing platforms and implementation trade-offs that inform ZY-Path. The CP1 themes—digital histopathology, pretrained models, local persistence and human-centred interaction—are retained. The CP2 review adds spatial sampling, preprocessing provenance and the distinction between software verification and model validation because these became consequential during implementation.

The review follows a thematic narrative approach, combining foundational research, platform publications and primary technical documentation. Each theme is linked to an implementation decision or evaluation requirement. Platform comparisons concern documented capabilities and their relevance to the selected task, with source limitations recorded in the reference audit.

2.2 Thematic Analysis of Related Research

2.2.1 Digital histopathology and the unit of analysis

Histopathology examines stained tissue to interpret cellular morphology and tissue architecture. H&E images provide patterns relevant to diagnosis and research, while digital analysis translates those images into computational inputs (Litjens et al., 2017). The unit of analysis must be explicit because an image patch, a larger raster region and a whole-slide pyramid have different sampling and resource requirements.

NCT-CRC-HE-100K contains 100,000 colorectal tissue patches across nine categories, while CRC-VAL-HE-7K provides a separate labelled evaluation collection (Kather et al., 2019). This project used the former for training and internal validation and the latter for the recorded 7,180-image test. The actual split manifests and measured results are described in Sections 3.3.2 and 4.2.6.

Dataset identity and pixel dimensions alone do not specify the physical field of view of an arbitrary imported raster image. Consequently, ZY-Path exposes source crop dimensions independently of network input dimensions. Section 4.5.3 discusses the resulting scale and generalisation limits.

2.2.2 Classification architectures and pretrained deployment

Image classification assigns a category to an image or patch, whereas semantic segmentation assigns categories to spatial locations. ZY-Path supports both tasks through separate model pathways. Residual networks, DenseNet and EfficientNet illustrate alternative classification designs based on residual learning, dense feature reuse and compound scaling respectively (He et al., 2016; Huang et al., 2017; Tan & Le, 2019).

DenseNet concatenates earlier feature maps into later layers to encourage feature reuse (Huang et al., 2017). The classifier trained for this project uses four dense blocks with two dense layers each. Its precise configuration, rather than a generic family name, governs checkpoint reconstruction and is reported in Section 3.3.2.

Architecture selection and application integration answer different questions. Residual networks and EfficientNet provide useful comparative foundations (He et al., 2016; Tan & Le, 2019), while ZY-Path concentrates on reconstructing supported models from metadata and presenting their outputs under explicit contracts. The project evaluates its chosen classifier rather than conducting an architecture benchmark.

Reusing a trained model for application inference is distinct from fine-tuning an ImageNet model. The classification training code directly instantiates DenseNet without loading pretrained ImageNet weights. I trained that configuration separately before integrating the selected checkpoint into the application; the GUI itself remains inference-only.

2.2.3 Semantic segmentation and multiscale context

U-Net established an influential encoder–decoder approach to biomedical segmentation with skip connections that preserve spatial information (Ronneberger et al., 2015). UPerNet instead combines multiscale feature processing for scene parsing; the current built-in integration reconstructs a UPerNet model with a Swin Base backbone through Transformers (Xiao et al., 2018; Liu et al., 2021). U-Net is relevant background, not the architecture of the built-in segmentation checkpoint.

Segmentation predicts categories on a spatial grid. UPerNet’s overlapping-window pathway combines class probabilities before selecting labels, then restores the

discrete mask with nearest-neighbour interpolation. This preserves label identities during restoration and provides the basis for the software's pixel-coverage statistics; predictive boundary quality requires separate annotated-mask evaluation.

2.2.4 Tiled classification, coverage and confidence

Spatial class maps can be constructed by assigning each crop's predicted class to its source region. This is useful for inspection but is coarser than segmentation. A class covering a large tile may occupy only part of that tile in the actual tissue. The resulting coverage must consequently be described as a tile-class estimate. Edge crops also matter: a smaller final crop must contribute fewer pixels to area coverage than a complete crop.

The model's softmax output is a normalised score over its classes. It is not automatically calibrated as the probability that an interpretation is correct (Guo et al., 2017). A high-confidence majority class across many tiles can arise from genuine image content, sampling scale, preprocessing mismatch or model/domain limitations. ZY-Path exposes per-tile probabilities and coordinates so these possibilities can be investigated rather than using a coloured map as proof of accuracy.

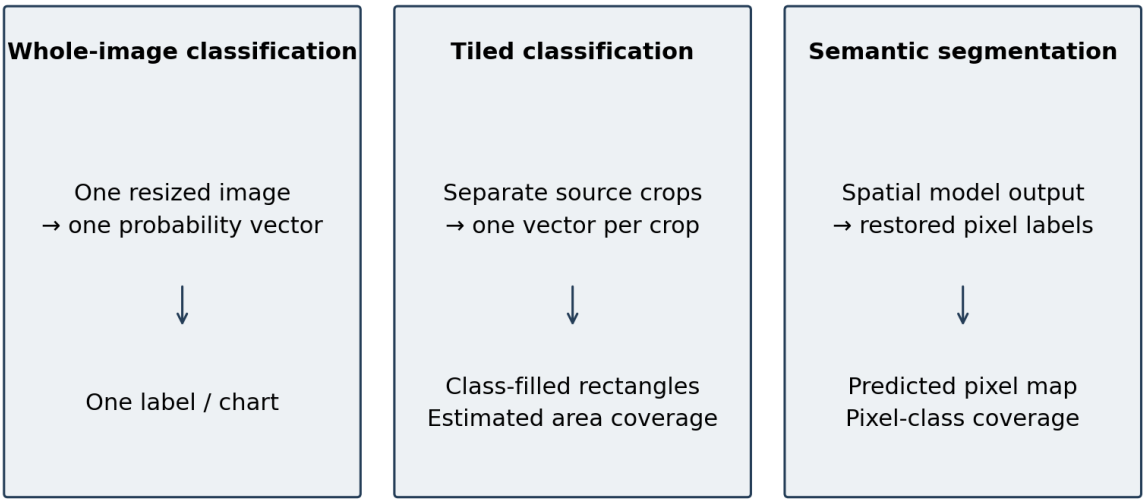


Figure 2.1. Three output meanings: whole-image classification, source-tile estimates and pixel segmentation. Author-created conceptual diagram; it contains no model accuracy results.

2.2.5 No-code interaction and transparent constraints

No-code deployment removes the need to write inference commands, but it does not remove the need for valid inputs. A .pth suffix cannot identify the network architecture, class order, normalisation or trustworthiness of a file. The interface must expose consequential choices and fail clearly when compatibility is not established. WSInfer's reusable-model approach provides an important precedent for model-aware graphical integration (Kaczmarzyk et al., 2024).

No-code interaction is implemented through explicit task controls and interpretable defaults. The mode, source dimensions and resize backend are visible before execution, and a stored record preserves the settings actually used. Separating model adapters, presentation and persistence also localises responsibility for changes, consistent with modular software design principles (Gamma et al., 1994).

2.2.6 Persistence and reproducible workflows

Persistent analysis records connect an image, model, task, parameters and saved output. For a single-user desktop workflow, an embedded database avoids administering a separate database service. SQLite’s implementation role is described once in Section 2.6; this section concerns the reproducibility requirements placed on the stored records.

A recorded configuration supports rerun only while its referenced files and compatible dependencies remain available. ZY-Path therefore uses the term recorded rerun for restoring the saved task, model and analysis parameters. Section 4.5.3 distinguishes this workflow continuity from immutable provenance.

2.2.7 Human-centred design and evaluation

Usability inspection and empirical user evaluation answer complementary questions. Inspection can expose inconsistent feedback or inaccessible controls, whereas task observation and questionnaires show how intended users actually experience the system (Yen & Bakken, 2012). Nielsen's heuristics support visible status, user control, consistency, error prevention and recognition rather than recall (Nielsen, 1994).

The System Usability Scale is a ten-item questionnaire for perceived usability (Brooke, 1996). Bangor et al. (2009) provide an interpretive framework relating SUS

scores to adjective ratings. These sources support the planned instrument and contextual interpretation, while task outcomes and qualitative observations explain which interactions influence the score.

SUS provides a standard ten-item instrument, but its 0–100 score is not a percentage of tasks completed or a measure of diagnostic accuracy (Brooke, 1996). Interpretation should include sample composition, missing data and the tasks users attempted. Feature-specific ratings and qualitative feedback can explain a SUS result, but should be analysed separately rather than added to the standard SUS calculation.

2.3 Review of Existing Solutions and Platforms

2.3.1 QuPath

QuPath is an open-source digital-pathology application with image visualisation, annotation, quantitative analysis and project-oriented workflows (Bankhead et al., 2017). It already provides a graphical interface and extensibility through scripts and extensions. Scripting is relevant to advanced customisation but is not a prerequisite for every interaction. Its whole-slide and annotation capabilities exceed the current scope of ZY-Path.

QuPath’s whole-slide, annotation and extensibility functions serve a broader image-analysis task. ZY-Path instead centres on supported local model files and individual analysis records. This is a comparison of workflow scope; the performance and usability questions require the empirical design discussed in Section 4.5.2.

2.3.2 Slideflow and Slideflow Studio

Slideflow supports deep-learning development for digital histopathology, with slide processing, model training and real-time whole-slide visualisation (Dolezal et al., 2024). Its official repository explicitly describes Slideflow Studio, model-development functionality, stain-processing tools and visualisation. These documented capabilities correct the proposal's implication that Slideflow is only a command-line framework.

Slideflow’s training and whole-slide capabilities address a broader research lifecycle. ZY-Path selects the inference-and-record portion of that lifecycle for individual users.

This trade-off motivates a task-based usability study rather than an assumption that fewer features necessarily produce a better interface.

2.3.3 WSInfer and QuPath integration

WSInfer makes pretrained deep-learning models reusable in a pathology workflow and integrates inference with QuPath (Kaczmarzyk et al., 2024). Its emphasis on model sharing and a standardised interface is directly relevant to the compatibility and provenance requirements of ZY-Path. It also demonstrates that no-code model use is not a unique invention of this capstone.

WSInfer provides a close precedent for accessible pretrained-model inference. ZY-Path's particular design combines explicit Single image/Tiled analysis controls, linked result inspection and local record retrieval/export. The comparison concerns this arrangement of functions within a raster-image application.

2.3.4 Cytomine

Cytomine supports collaborative analysis of very large images, image annotation and integration of analysis workflows (Marée et al., 2016). A collaborative web architecture is appropriate where shared images and multi-user annotation are central. It introduces infrastructure and administration considerations that differ from a single-user local desktop application.

ZY-Path trades these collaborative and large-image-management capabilities for a local application with no running application server. That trade-off is a scope choice. A local SQLite database does not replace a collaborative repository, access-control system or institutional image-management infrastructure.

2.3.5 Commercial environments

Commercial platforms such as Aiforia Create provide an alternative model-development and deployment environment (Aiforia, n.d.). Their value can include managed workflows and institutional support, while licensing, deployment and import interfaces depend on the product and agreement. This project did not obtain licensed installations or independently test their supported model formats.

For this project, local control of the source code, model selection and analysis records is the relevant deployment choice. Product-specific licensing, import and

support arrangements require current primary documentation; they are not treated as universal disadvantages of commercial systems.

2.4 Comparative Evaluation of Existing Systems

Table 2.1 compares platform purposes and documented workflow scope. It separates an unexamined capability from an absent one and uses no aggregate ranking because the applications address different tasks. Source publications are cited in Sections 2.3.1–2.3.5.

Table 2.1. Scope-based comparison of pathology platforms

System	Documented emphasis	Interface/deployment	Relation to ZY-Path
QuPath	Whole-slide viewing, annotation, quantitative analysis and extensions	Graphical desktop; optional scripting	Broader image-analysis scope; advanced extension setup differs from a constrained model-file workflow
Slideflow/Studio	Model development and whole-slide visualisation	Python research environment plus graphical Studio	Includes training and WSI capabilities outside this project's scope
WSInfer + QuPath	Reusable pretrained pathology inference	Inference integrated with a desktop pathology viewer	Closest precedent for graphical model use; model-distribution and WSI focus differs
Cytomine	Collaborative large-image annotation and analysis	Web/server-oriented collaboration	Supports multi-user work excluded from ZY-Path
Aiforia Create	Commercial AI model development/deployment	Vendor-managed product options	Import, licensing and history details not independently evaluated
ZY-Path	Supported local inference, spatial inspection, record restoration and export	PyQt5 desktop with local CPU and SQLite	Focused raster-image workflow; classifier evaluated on 7,180 test images; UAT and package validation pending

Source: platform publications and product documentation cited in Sections 2.3.1–2.3.5; ZY-Path scope from the project implementation.

Two considerations follow. First, classification/segmentation support alone is insufficient for comparing workflows: users also need to understand sampling scale,

probability meaning and provenance. Second, a lighter local workflow can be useful without replacing an established WSI platform. ZY-Path's evaluation should therefore test its own requirements before making comparative claims.

2.5 Identified Research Gaps and Project Positioning

The project addresses three scoped integration needs. The first is model-aware accessibility: compatible models should be usable through graphical controls while unsupported inputs are rejected with actionable messages. The second is interpretive consistency: source, overlay, class map, per-tile detail and aggregate statistics should use the same spatial coordinates and explicit task semantics. The third is record continuity: history restoration, rerun and export should retain the analysis configuration rather than current defaults.

The resulting project position is a focused desktop integration for individual educational and research use. Model training and patch-level evaluation support the classification component, while the application contribution connects compatibility, spatial interpretation and record continuity as observable requirements.

The positioning also identifies an evaluation gap within this project itself. Automated tests can verify tile coordinates and metadata restoration, but they cannot establish that users understand the difference between coverage and confidence. The planned UAT therefore includes explicit interpretation tasks in addition to ordinary button completion.

2.6 Technological Infrastructure and Implementation Considerations

Python remains the implementation language because it connects the desktop GUI, tensor inference and scientific imaging libraries. The CP1 three-layer concept remains useful: presentation is implemented in ui/, inference in core/, and persistence/report generation in data/. MainWindow also coordinates saving and display, so the design is a layered desktop application rather than a perfectly isolated MVC implementation.

SQLite stores structured records in an embedded, serverless database, making it suitable for a local application without a separately administered database service (Kreibich, 2010; SQLite, n.d.). In ZY-Path it links inference identity and timestamps to

task-specific metadata and saved artifacts. Parameterised operations and transactions provide the database component of persistence, while external artifact handling remains the coordinator's responsibility.

PyQt5 supplies windows, layouts, signals and worker threads (Riverbank Computing, n.d.; The Qt Company, n.d.). QGraphicsView provides source-coordinate transformations, linked navigation and vector grid painting. PyTorch supplies model execution and inference_mode (Paszke et al., 2019); NumPy and Pillow support image conversion (NumPy Developers, n.d.; Pillow Contributors, n.d.). Matplotlib renders the Single image probability chart (Hunter, 2007), and fpdf2 generates structured PDF reports (fpdf2 Contributors, n.d.). These choices connect the interface to a local scientific-Python stack without a web backend.

The dependency declarations specify PyQt5 5.15.x, PyTorch 2.1.x, torchvision 0.16.x, Transformers 4.38.2, Pillow 10.x, NumPy 1.26.x, Matplotlib 3.8.x, OpenCV-headless 4.11.0.86 and fpdf2 2.7.x. These are declared ranges or pins, not a complete environment lock. OpenCV is used only for the explicitly selectable classifier resize backend; Pillow bilinear remains the default. No REST API, web frontend or remote inference service is required by the implemented application.

UPerNet reconstruction loads its configuration from the local Hugging Face cache with local_files_only=True. Preparing a redistributable application therefore requires packaging the permitted weights, cached configuration and runtime dependencies. PyInstaller is a candidate tool for this future deployment step (PyInstaller Development Team, n.d.); the current deployment evidence is assessed in Section 4.3.4.

2.7 Synthesis of Findings and Project Implications

The literature supports pretrained-model reuse, explicit task semantics and complementary technical/user evaluation. It also demonstrates that graphical pathology systems and sophisticated visualisation already exist. ZY-Path should consequently be judged on correct integration and workflow clarity rather than unsupported novelty or superiority claims.

Three implications guide implementation. Model metadata must govern preprocessing and labels. Spatial presentation must preserve coordinate meaning

across Fit, zoom, comparison and restoration. Persistent records must keep the actual analysis configuration, while communicating their dependence on files and environment stability. These implications are translated into the requirements and traceability matrix in Chapter 3.

2.8 Summary

The reviewed work establishes the conceptual basis for a local inference application and clarifies its boundaries. The final implementation retains the CP1 emphasis on a no-code interface, local persistence and result export, while adding explicit tiled classification and stronger presentation/provenance controls. The next chapter explains how these design choices are implemented and how their available evidence is evaluated.

Chapter 3: Methodology

3.1 Overview of System Methodology

I developed ZY-Path through iterative implementation and regression-driven refinement of the CP1 design. Each iteration connected a requirement or observed defect to the affected code, an implementation change and an appropriate regression check. Observations during Windows testing informed interface corrections, while automated Qt tests made those cases repeatable.

The implemented system has three logical layers. The presentation layer contains the main window, inference/history/action panels, settings and result viewers. The core layer loads models, prepares inputs, executes inference and parses outputs. The data layer stores records and creates export files. MainWindow coordinates transitions between these layers, including saving result artifacts before inserting a completed history record.

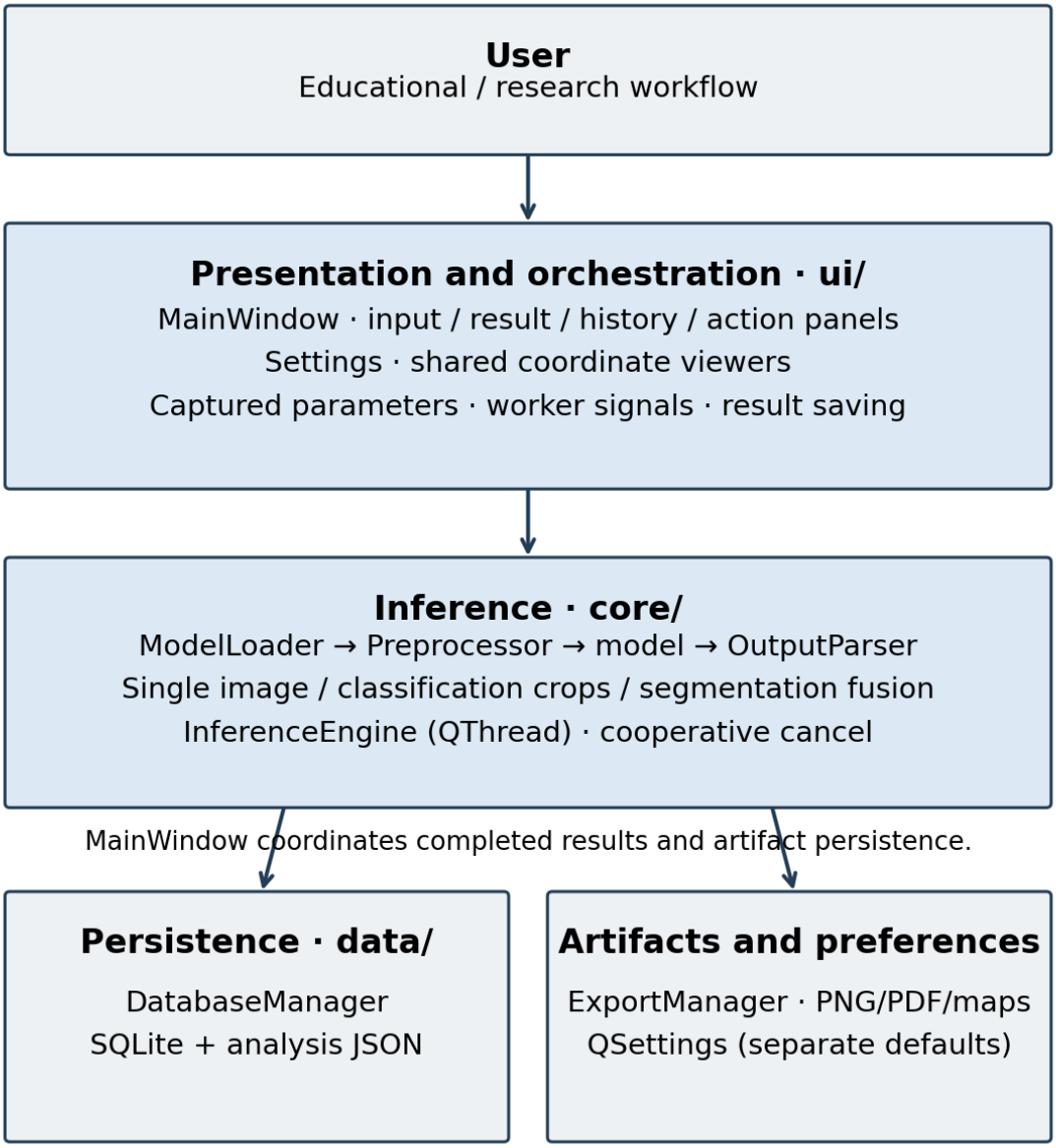


Figure 3.1. Implemented logical architecture and artifact flow. Source: author, based on the project's Python modules.

The implementation baseline is revision 1f3e01e, supplemented by the lifecycle and responsive-layout repairs evaluated in the final Windows run. Classification methods are documented by the training code provided by my supervisor, my training/validation records and the corresponding test outputs. The segmentation pathway follows the model and reference inference notebook provided by my supervisor. Appendix C identifies the evaluated source state and the evidence supporting each result.

3.2 Functional and Non-Functional Requirements

3.2.1 Functional Requirements

The requirements preserve the original model–image–inference–history workflow and add explicit spatial analysis and state-management requirements revealed during development. A requirement describes intended behaviour; its implementation and evidence status are recorded separately in Section 3.6.6.

Table 3.1. Functional requirements and acceptance conditions

ID	Requirement	Observable acceptance condition
F01	Load supported raster images	Selected input is identified and rendered; invalid files produce a clear error
F02	Load and validate trusted compatible models	Supported representations load; finite output/task/class checks reject unsupported inputs
F03	Run single-image classification	The complete selected image is processed once and yields ordered class probabilities
F04	Run explicit tiled classification	Selected width/height generate non-overlapping real crops, including partial edges
F05	Show classification statistics	Nine-class legend, per-tile label/confidence and actual-area estimates agree with stored tiles
F06	Run compatible segmentation	The segmentation model produces a restored label map, overlay and its own legend/counts
F07	Inspect spatial results	Overlay/map/original, opacity, comparison, linked navigation and appropriate grid remain aligned
F08	Save completed history	A record connects actual model/task/mode/parameters to saved artifacts
F09	Search and filter history	Keyword and inclusive local-date queries return the expected records
F10	Restore a stored result	Restoration displays the recorded result without initiating a new forward pass
F11	Rerun a record	The recorded model, image, task and parameters take precedence over current defaults
F12	Export results	PNG/PDF contain the selected presentation and available recorded metadata
F13	Cancel inference	Cooperative cancellation leaves no new completed record and preserves the previous result

ID	Requirement	Observable acceptance condition
F14	Configure task-specific model defaults	A validated local default can be saved/restored independently for each task
F15	Configure analysis and interface preferences	Mode/default selection, source tile dimensions and resize method persist with clear scope
F16	Manage records safely	Confirmed deletion removes selected records/app-owned artifacts without deleting source inputs

Source: author's requirements derived from the CP1 objectives and implemented workflows.

3.2.2 Non-Functional Requirements

Table 3.2. Non-functional requirements and verification boundaries

ID	Quality requirement	Verification approach	Present boundary
N01	Responsive interaction	Worker-thread signals; cancellation and close-lifecycle tests	A model operation may delay cancellation acknowledgement
N02	Traceable analysis	Recorded parameters, restored metadata and rerun isolation tests	File hashes and complete environment snapshots are not persisted per run
N03	Local data handling	Code-path inspection; local SQLite/artifact storage	Local files are not automatically encrypted or backed up
N04	Readable/resizable interface	Small logical screens, font/DPI simulation, theme and focus tests	Native Windows automated layout/window-state checks passed; complete device/accessibility manual acceptance pending.
N05	Measurable runtime/resource use	Planned fixed-workload benchmark with CPU/RAM/image/tile details	No independent performance benchmark is available
N06	Predictable error handling	Missing-file, invalid-model, export/database and cancellation tests	Structural model validation is not a security or accuracy certificate

ID	Quality requirement	Verification approach	Present boundary
N07	Deployable offline runtime	Local configuration/weights and clean-machine packaging checks	Real classifier loads on Windows; complete offline package and clean-machine deployment unverified
N08	Maintainability and regression control	Layered modules, named tests, source revision and preserved failure reporting	No current coverage percentage is claimed

Source: author's quality requirements and recorded verification scope.

The CP1 target of at least 80% coverage of core/data remains an unmeasured target because the recorded pytest commands did not generate a coverage report. Requirements therefore retain separate implementation and verification status rather than inferring coverage from the number of passing tests.

3.2.3 Ethical Considerations

The application is an educational and exploratory research tool. It does not issue medical diagnoses, recommend treatment or replace professional review. Labels such as TUM and Cancerous epithelium are model categories, not verified findings about an individual patient. Softmax confidence is displayed as model output and must not be presented as a clinically calibrated risk estimate.

Demonstrations and UAT should use authorised, de-identified images. Source paths and exported metadata require inspection as well as the visible image. Participant consent and institutional requirements will be recorded with the UAT materials in Appendix B.

Local execution avoids uploading images to an inference service, but it does not automatically provide encryption, secure deletion, access control or regulatory compliance. History files, exported reports and backups remain subject to the host computer's security arrangements. Reports intended for sharing should be inspected for identifying paths and metadata.

Model trust is a further ethical and technical concern. The loader first attempts supported loading paths, including a weights-only torch.load attempt, but retains a fallback for serialized modules. Such files can execute code during deserialisation.

The intended workflow therefore accepts trusted models from known sources; a file-extension filter and dummy forward pass must not be described as a sandbox or malware detector (PyTorch Contributors, n.d.).

3.2.4 Accessibility Compliance

Accessibility is addressed as a set of design and verification activities. Controls use text labels and tooltips; class legends include names and numerical values as well as colour; the interface supports keyboard focus and Space toggling for result checkboxes; scroll areas and draggable splitters expose content in smaller windows. High-DPI sizing uses Qt logical coordinates rather than multiplying screen geometry by the device pixel ratio.

WCAG 2.2 informs the review of keyboard access, contrast and non-colour cues, adapted to a desktop interface (W3C, 2023). Native assistive-technology testing and a complete contrast/accessibility audit are further evaluation activities; their present status is included in Section 4.5.3.

The nine-class palette is accompanied by names, counts and numerical statistics so interpretation does not depend on colour alone. Stable control geometry and a visible keyboard focus cue support mouse and keyboard interaction. These design choices are evaluated through controlled interaction tests and the planned user study.

3.3 Methodologies, Tools, and Techniques

3.3.1 Model representations and executable validation

ModelLoader accepts .pt/.pth files subject to existence, non-empty-file and maximum-size checks. The maximum is 500 × 1024² bytes, or 500 MiB. It attempts TorchScript, serialized nn.Module objects and supported checkpoint dictionaries. Named torchvision reconstruction is limited to the architectures explicitly registered in the loader; a general state dictionary with no recognised architecture contract is not arbitrarily guessed.

The CRC checkpoint contains DenseNet configuration, class labels, raw255 preprocessing and a 224 × 224 input size. The UPerNet state-dictionary path reconstructs the nine-class Swin Base configuration from the local Hugging Face

cache and loads the weights strictly. Missing configuration or incompatible state shapes produces an actionable validation error.

Validation executes a float32 RGB probe of shape (1, 3, 224, 224), extracts a tensor or an object's logits field, rejects non-finite values and identifies classification output (1, C) or segmentation output (1, C, H, W). It checks executable structure, not training quality, authenticity, calibration or diagnostic fitness. Models with special input requirements, unsupported dictionaries or missing custom Python classes may still be incompatible even when their file extension is allowed.

3.3.2 Classification training and model configuration

Using the reference training code provided by my supervisor, I trained torchvision.models.DenseNet with growth_rate=32, block_config=(2, 2, 2, 2), num_init_features=64, bn_size=4, drop_rate=0 and num_classes=9. The training logs report 416,457 parameters. The class order is ADI, BACK, DEB, LYM, MUC, MUS, NORM, STR and TUM.

The training loader decodes images with OpenCV, converts BGR to RGB and applies Albumentations Resize(224, 224), vertical/horizontal flips, hue adjustment, rotation and a random-sized crop. Validation uses Resize followed by ToTensorV2 and float conversion. Values remain on the raw255 scale. Training used class-weighted cross-entropy, Adam with initial learning rate 0.001 and weight decay 10^{-6}, and ReduceLROnPlateau with factor 0.5, patience 3 and minimum learning rate 10^{-6}; batch size was 32 with two data-loading workers.

The seed-42 manifests partition NCT-CRC-HE-100K into 80,002 training and 19,998 validation images using stratified patch-level sampling. Manifest file hashes agree with the experiment configuration, and their relative image paths are disjoint. Training continued to 30 epochs despite the initial experiment directory name ending in 10ep; the checkpoint at epoch 26 had the lowest recorded validation loss. The training logs identify an NVIDIA L4, distinct from the CPU-only Windows application environment.

The formal nine-class run is identified by its manifests, completed metrics, checkpoint and test-7k outputs. Older three-class scripts and smoke-check experiments remain development materials and are not used to establish the CRC

results. Training/validation file-path separation is verified; patient- or slide-level isolation requires grouping information not present in those manifests.

3.3.3 Classification preprocessing and resize provenance

Preprocessor converts each input to RGB and outputs a contiguous float32 NCHW tensor. For the built-in classifier, the tensor shape is (1, 3, 224, 224) and the profile is raw255: pixel values are kept on their original approximately 0–255 scale. The optional imagenet profile divides by 255 and applies mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). It is not silently substituted for the built-in raw255 profile.

The default classification resize backend remains pillow_bilinear. The optional opencv_linear backend explicitly calls cv2.resize(..., interpolation=cv2.INTER_LINEAR), retains uint8 interpolation rounding and then converts to float32. Both single-image and tiled classification use the selected backend through the same Preprocessor implementation. The backend name is stored with the analysis so rerun does not depend on a later preference change.

The original evaluation used Albumentations 1.3.1 and OpenCV 4.8.1, with A.Resize(224, 224) retaining its default linear interpolation. Pillow bilinear and OpenCV INTER_LINEAR can differ when resampling is necessary. A Windows rerun through the default Pillow pathway is now reported in Section 4.2.6; all 7,180 source patches were already 224 × 224, so this run validates the native-sized patch pathway rather than general equivalence when larger crops are resized.

3.3.4 Explicit classification modes and crop geometry

Single image resizes the complete selected image and obtains one class-probability vector. Tiled analysis instead partitions source coordinates into non-overlapping rectangles. For image width W, height H and selected source tile width w and height h, the number of tiles is:

$$N = \text{ceil}(W / w) \times \text{ceil}(H / h). \quad (3.1)$$

At source location (x, y), the crop rectangle is [x, y, min(x+w, W), min(y+h, H)]. The right and bottom coordinates are exclusive. Each rectangle is cropped from the actual source image, independently resized to network input and classified once.

Partial edge crops contain real pixels only; they are neither padded nor replaced by the complete image or a previous crop.

For W=3552, H=3536 and w=h=512, Equation 3.1 gives seven columns and seven rows, or 49 tiles. The final column is 480 pixels wide, the final row 464 pixels high, and the bottom-right area 222,720 pixels. A deterministic fixture verifies these coordinates and complete non-overlapping coverage.

Source crop dimensions determine the field of view before resampling to 224 × 224. The interface keeps this choice explicit and labels the 512 × 512 starting value as unconfirmed relative to training scale. The evaluation consequences are discussed in Section 4.5.3.

3.3.5 Class probabilities, maps and estimated coverage

For tile i with class logits z_i, the softmax probability for class c is $p_ic = \exp(z_ic) / \Sigma_j \exp(z_{ij})$. The tile label is $k_i = \text{argmax_c } p_ic$ and its displayed confidence is p_i,k_i. These values are retained alongside the source rectangle. The ordered nine-class colour corresponding to k_i fills that crop's source coordinates in the classification map.

Let a_i denote the actual crop area. The tile count for class c is $n_c = \Sigma_i 1(k_i=c)$, while its estimated covered area and ratio are:

$$A_c = \Sigma_i a_i 1(k_i=c), \qquad r_c = A_c / (W \times H). \qquad (3.2)$$

The denominator includes the complete source image, including crops assigned to background or other classes. Counts and coverage answer different questions. A small edge crop contributes one tile to n_c but only its real area to A_c. The ratios sum to one for a complete partition, apart from numerical rounding in display.

The implementation also retains an aggregate class based on area-weighted mean probabilities:

$$q_c = \Sigma_i a_i p_ic / (W \times H), \qquad k^* = \text{argmax_c } q_c. \qquad (3.3)$$

This aggregate need not equal the class with the largest r_c because probability mass and hard-class coverage are different summaries. It is not a new whole-image forward pass. The history table therefore shows tile count and largest estimated coverage, leaving whole-image confidence blank; PDF metadata distinguishes “Aggregate tile label” and “Area-weighted mean probability”.

Table 3.3. Classification class order and fixed display palette v1

Index	Label	Tissue category	RGB
0	ADI	Adipose	(230, 159, 0)
1	BACK	Background	(210, 210, 210)
2	DEB	Debris	(86, 180, 233)
3	LYM	Lymphocytes	(0, 158, 115)
4	MUC	Mucus	(240, 228, 66)
5	MUS	Smooth muscle	(0, 114, 178)
6	NORM	Normal colon mucosa	(213, 94, 0)
7	STR	Stroma	(204, 121, 167)
8	TUM	Tumour epithelium	(120, 60, 140)

Source: classification checkpoint class order and the application's classification display palette; colours are display metadata.

The palette is application display metadata, not evidence of a training colour convention. ClassificationTileSpec validates source/input dimensions, nine unique model classes, matching class order, matching preprocessing and nine distinct RGB colours. An incompatible class contract is rejected rather than substituted with segmentation labels.

3.3.6 Segmentation pathway and label restoration

My supervisor provided the segmentation model and the reference notebook predict-upernet-tme-tissue-overlap.ipynb; I did not participate in segmentation training. The notebook constructs a nine-label UPerNet using the openmmlab/upernet-swin-base configuration. Its large-image example uses source tile=1000 and stride=512, which are the application defaults; network input is 224 × 224. The application runs the path on CPU with batch size one, while the notebook example requests CUDA and its function defaults to batch size eight.

For the UPerNet pathway, the final window on each axis is aligned with the image edge. Images smaller than a source tile are reflect-padded. Each RGB patch is resized with Pillow bilinear, scaled to 0–1 and normalised with ImageNet mean

(0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). Softmax probabilities are accumulated on a reduced-resolution canvas with a two-dimensional Hann window plus 0.001, normalised by accumulated weights, converted to labels and restored by nearest-neighbour interpolation before cropping to the original dimensions.

The resulting nine-label counts use the final restored label map. Pixel coverage is class pixel count divided by W × H, including Exclude and Whitespace/Empty in the denominator. It is a statistic of a predicted mask, not an editable estimate of ground truth. The output retains its actual tile/stride/input metadata and its own colour semantics.

Table 3.4. Built-in segmentation labels and display colours

Index	Model label	RGB
0	Exclude	(0, 0, 0)
1	Cancerous epithelium	(255, 0, 0)
2	Stroma	(0, 255, 0)
3	TILs	(0, 0, 255)
4	Normal epithelium	(255, 255, 0)
5	Junk/Debris	(128, 0, 128)
6	Blood	(0, 255, 255)
7	Other	(255, 165, 0)
8	Whitespace/Empty	(255, 255, 255)

Source: supervisor-provided segmentation notebook; labels and RGB values match the application constants.

The nine-label order and RGB colours in Table 3.4 match the reference notebook. Generic compatible segmentation models use a separate prepared-tensor and OutputParser pathway. The model filename’s 100ep and panoptils strings are retained as identifiers; training duration and dataset provenance are not inferred from those strings.

3.3.7 Background execution and cancellation

InferenceEngine subclasses QThread and emits result_ready, error_occurred, progress_changed, status_changed and cancelled signals. ModelValidationThread performs asynchronous validation. The GUI captures the intended task and parameters before starting a worker, locks conflicting controls while busy and updates status/progress through signals rather than running the forward pass in the GUI event loop.

Cancel sets a threading.Event. The worker checks it before/after key stages and between tile operations; it cannot interrupt a running PyTorch forward operation at an arbitrary instruction. The interface therefore distinguishes the request ("Cancelling...") from acknowledgement, keeping inputs locked until the worker exits. A late result after cancellation is rejected, and no partial completed record is saved. Closing while a worker or model-validation task is active waits for cleanup rather than terminating the thread forcibly.

A final lifetime guard also waits for an active worker before Qt destroys its owning widget. Model-validation signals connect to QObject-bound receivers, so destruction disconnects pending callbacks instead of allowing them to access deleted controls. Regression cases cover forced parent deletion and both successful and failed validation signals for built-in models, custom models and task-default settings.

3.3.8 History schema, artifacts and rerun isolation

DatabaseManager uses sqlite3 connections with parameterised statements and transaction handling. The base inference_records table stores paths, task, label/confidence where applicable, result-image path and timestamp. A companion segmentation_artifacts table stores artifact paths and analysis_json; despite its legacy name, it also carries tiled-classification and single-classification parameter metadata.

Table 3.5. Implemented persistent data structures

Structure	Principal fields	Purpose
inference_records	id, image_path, model_path, task_type, result_label, confidence, result_image_path, timestamp	Searchable completed-analysis identity and base metadata

Structure	Principal fields	Purpose
segmentation_artifacts	record_id, result_labels_path, result_color_mask_path, result_overlay_path, analysis_json	Optional spatial artifacts and recorded analysis/presentation metadata
Classification analysis JSON	mode, resize_backend, classes/colours, source/input sizes, tiles/probabilities, counts, areas, scale_status, presentation	Preserve tile semantics and restore/reuse the actual parameters
Segmentation analysis JSON	recorded task/model, labels/colours, dimensions, tile/stride/input, counts, presentation	Restore spatial results without borrowing classification semantics
QSettings	theme, current/default classification settings, per-task default model paths, reminders, window geometry	User preferences, distinct from completed record parameters

Source: project database schema, task metadata and QSettings implementation.

UTC timestamps are stored and converted for local display/filtering. A date range is interpreted as inclusive local days and converted to UTC query boundaries. Both date selectors and popup calendars explicitly use English, while the date format remains yyyy-MM-dd. Searches cover image/model names and do not perform visual image-similarity retrieval.

Restoring a record reads its saved artifacts and metadata without rerunning the model. Rerun is a separate operation that selects exactly one recorded analysis and uses its model, source image, task, mode, tile dimensions and resize backend. Missing files produce a clear message rather than an unannounced substitution. Current model defaults and preferences do not relabel an existing record. Deletion requires confirmation and removes selected records plus recognised app-owned result artifacts, not the original user image or model.

Saving is not a distributed transaction across image files and SQLite. Artifacts are written through the application coordinator and the record is inserted when saving succeeds; failures are reported. Filesystem errors can still leave incomplete or orphaned artifacts, and backups are not automatic. This is a maintenance/reliability boundary rather than an implemented guarantee of atomic recovery across all resources.

3.3.9 Export and local model defaults

ExportManager uses Pillow/Matplotlib for PNG and fpdf2 for structured reports. Spatial exports can include analysis JSON, class/label maps and overlays as appropriate to the recorded task; a PDF includes available source/result images and metadata. The report uses the current result's recorded identity, not whichever model happens to be selected for a subsequent run. Changing overlay opacity or view mode changes presentation, not class labels, probabilities or stored counts.

The Models settings page maintains independent classification and segmentation defaults. A candidate default is validated for the intended task before it is saved, and “restore built-in” resets that task's model preference. A missing custom default is reported as unavailable rather than silently falling back. Recorded rerun uses the saved model path. Defaults are convenience preferences, not a migration of earlier analyses to new weights.

3.4 User Interface (UI) and User Experience (UX) Design

3.4.1 Principles and information hierarchy

The interface keeps the CP1 three-region structure: input controls on the left, analysis/history in the centre and export/session actions on the right. Implementation changes replace fixed panel assumptions with draggable splitters, scrollable contents and bounded window restoration. The aim is to keep essential actions reachable while prioritising the analysis image.

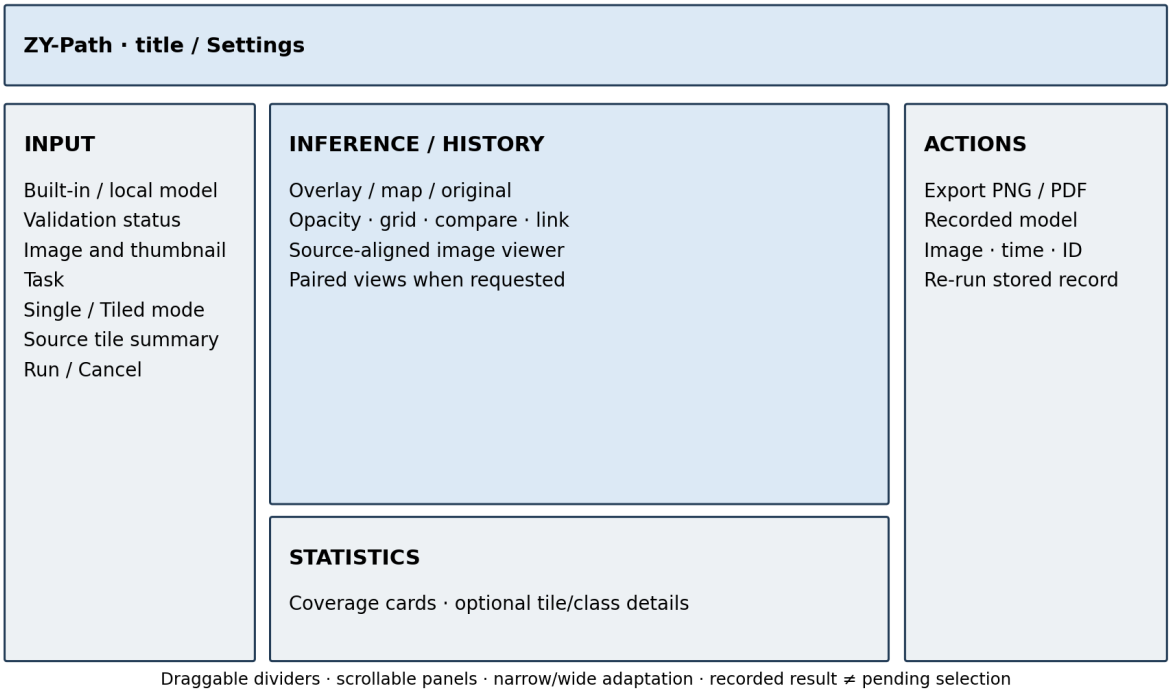


Figure 3.2. Annotated final workflow wireframe. This author-created design diagram summarises implemented regions and is not a Windows runtime screenshot.

Task and classification mode are visible before running. Status messages distinguish validation, inference, cancellation and completion, while completed-result metadata remains separate from a pending selection. These decisions apply visibility, consistency, recognition and user control from the usability literature (Nielsen, 1994; Yen & Bakken, 2012); their effect on perceived usability will be assessed through the planned UAT.

3.4.2 Spatial results and progressive disclosure

Tiled classification and segmentation initially hide the duplicated source preview so the main analysis view has more space. Users can select Original image or enable Compare with original. Overlay is the default spatial presentation; opacity is adjustable, and a pure class/segmentation map remains available. The classification coverage cards stay visible without requiring navigation among Image, Legend and Tiles pages; per-tile and extended class details are disclosed when needed.

In comparison mode, linked views share zoom and source-coordinate centres. Both panes use a common Fit scale rather than independently fitting dissimilar viewport widths. Narrow pages stack the pair vertically; widening restores horizontal comparison. When segmentation comparison is active, its legend stays below the

images to avoid a wide-window side legend shrinking the comparison back below its horizontal-layout threshold.

The classifier grid is painted after the image with the same scene-to-viewport transform and an approximately constant logical line width. This avoids losing lines by pre-rendering a fine grid into a large image and then shrinking it. Fit, 1:1, manual zoom, resize and history restoration preserve source-coordinate alignment. The displayed scale accounts for device pixel ratio: 1:1 denotes one source pixel per physical display pixel, while window bounds are handled in Qt logical coordinates.

3.4.3 Control states, settings and help

Result toggles use a padded rounded surface. Their complete visible area—including label and padding—accepts a mouse click. Mouse focus does not leave the former thick blue frame; keyboard Tab/Backtab focus retains a thin accent border, and Space toggles the state. Constant border width prevents focus/checked states from changing the control's size.

Settings is resizable and scrollable. Preferences groups export reminders, classification mode, source tile dimensions, resize backend, separate colour references and layout reset. Selecting a mode applies to the main panel and startup default, while history restoration does not rewrite that default. Closed dropdowns pass wheel gestures to page scrolling rather than changing the selection accidentally.

Guide & Support provides numbered guidance, persistent Contents navigation, previous/next actions and a reading-aware indicator. System Information displays runtime and dependency information before copying it, with confirmation expiring after 2.5 seconds. About reports version 1.0. Together these controls make configuration and support information discoverable within the application.

3.4.4 Responsive windows and platform limits

Window fitting uses the current screen’s availableGeometry in logical coordinates and checks restored size/position. The implementation distinguishes normal, maximised and fullscreen states and avoids continuously forcing a size during interactive resize. Inner scroll areas may shrink vertically while Run, Cancel and Re-

run retain usable space; their preferred content heights must not stretch the entire three-panel splitter beyond its viewport.

Automated checks cover small logical screens, repeated narrow/wide transitions and Qt scale-factor simulation. The final native Windows suite also passed the maximise/restore, saved-geometry, comparison/grid and compact-viewport cases. Formal end-to-end acceptance with authorised real images remains a separate activity in Appendix D.

3.5 Unified Modeling Language (UML) Diagrams

3.5.1 Use Case Diagram

The primary actor is an educational/research user. No administrator or clinical approval role is implemented. Model validation is included in the run workflow; tiled classification extends the classification choice when that explicit mode is selected. Restoring history and rerunning history are separate use cases because only the latter executes inference and creates a new record.

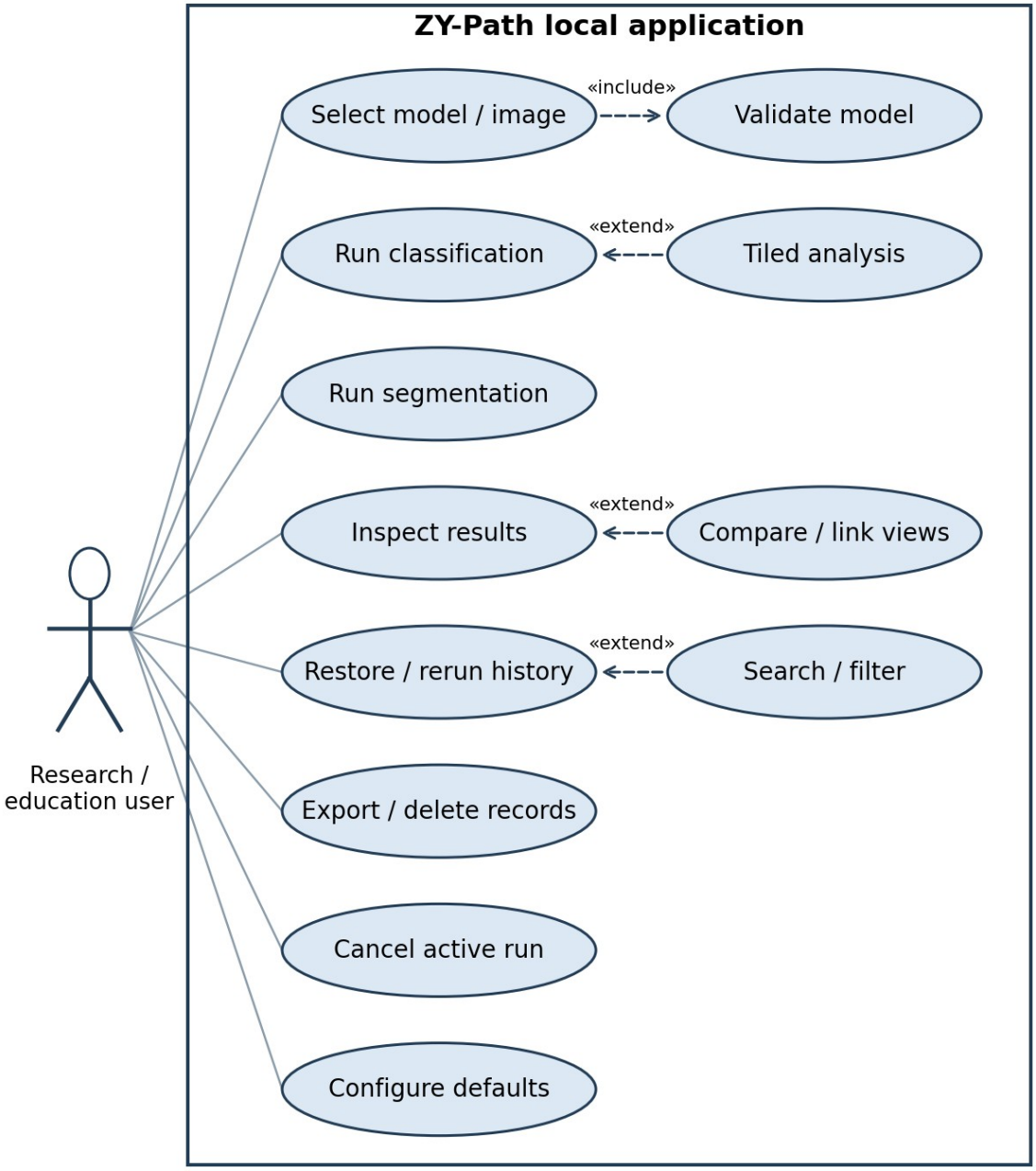


Figure 3.3. Use case diagram for the implemented local analysis workflow. File/model trust and clinical interpretation remain external responsibilities. Source: author, based on the implemented application.

3.5.2 Sequence Diagram

The sequence diagram follows a successful completed run after the user selects an image, task and model. MainWindow captures parameters, starts the worker and receives progress/result signals. The worker validates and executes the pipeline; the GUI coordinator renders the result, saves artifacts and inserts a history record.

Cancellation and saving failure are alternatives, not successful completed-history paths.

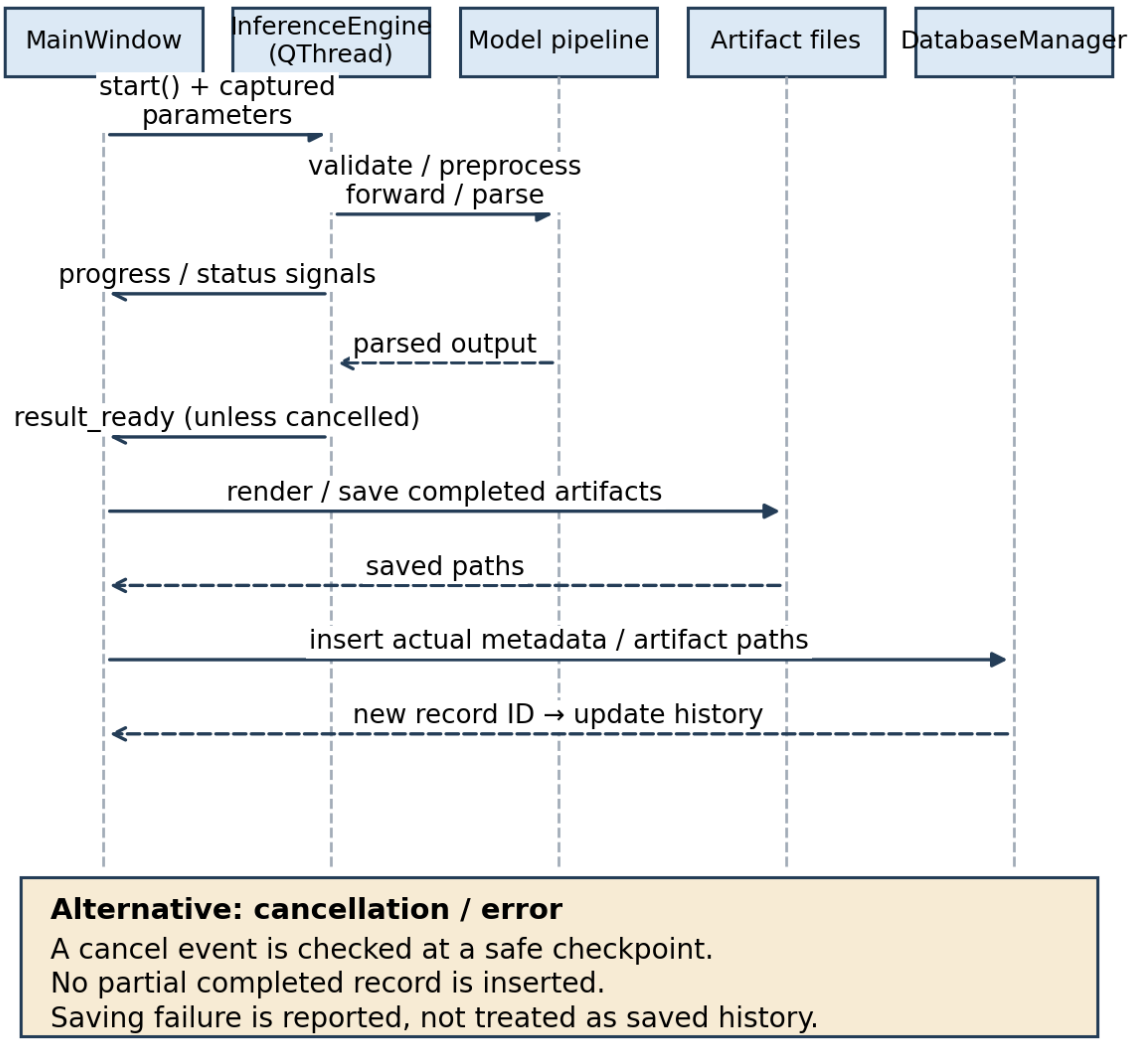


Figure 3.4. Sequence diagram for a run and its persistence boundary. Cancellation is acknowledged after a safe checkpoint; artifact-save failures must not be represented as successful saved records. Source: author, based on the implemented application.

3.5.3 Class Diagram

The class diagram shows principal runtime responsibilities rather than every widget. MainWindow coordinates panels and services; InferenceEngine depends on the loader, preprocessor, parser and task-specific spatial functions. ClassificationResult and SegmentationResult share ImageComparison and ZoomableImageView. DatabaseManager and ExportManager handle persistence/export; QSettings stores preferences separately.

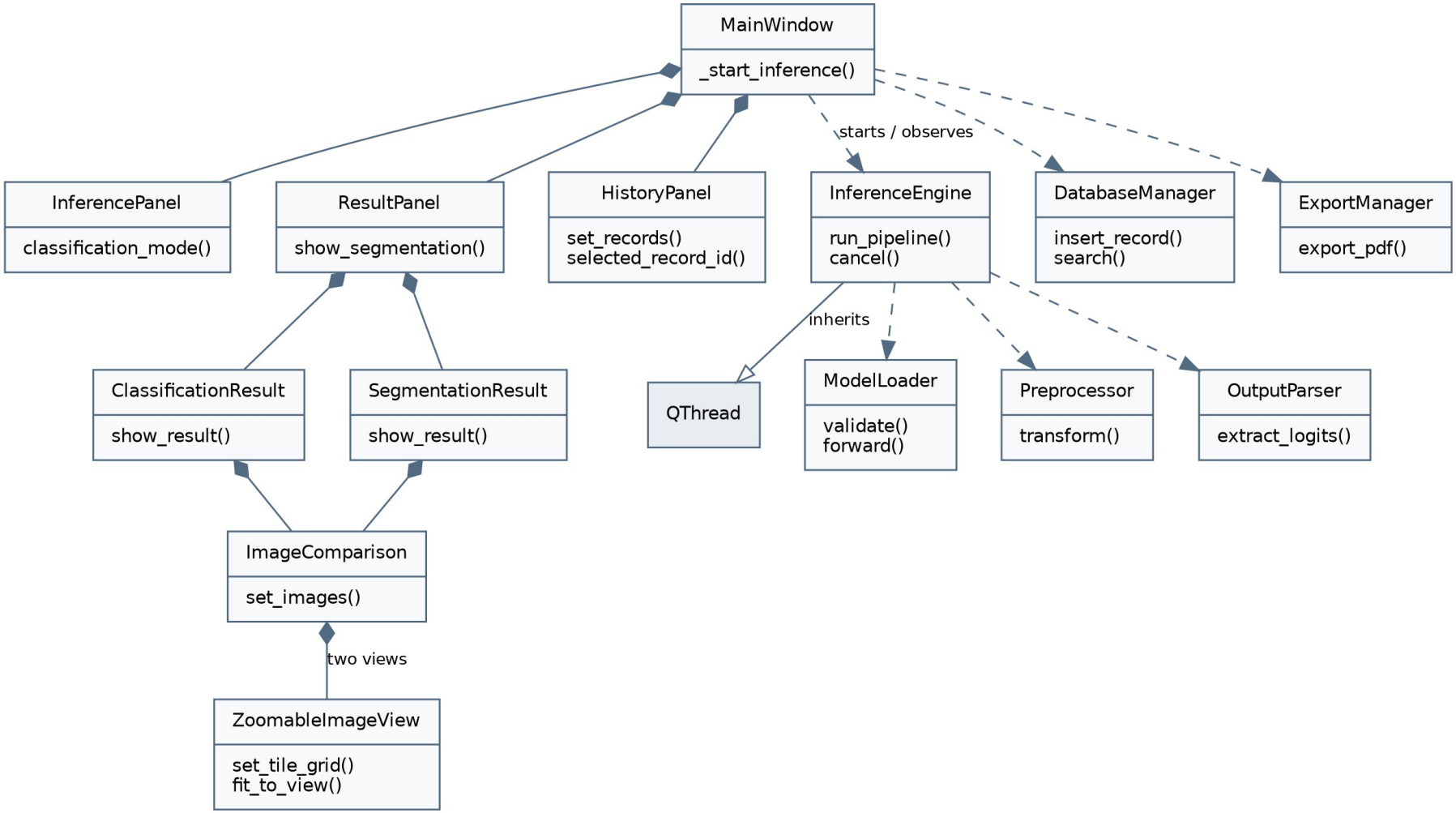


Figure 3.5. Implementation-level class diagram. Relationships represent principal ownership or dependency, not a generated inventory of every method. Source: author, based on the implemented application.

3.5.4 Activity Diagram

The activity diagram highlights the explicit mode decision, distinct preprocessing/task branches and completion/cancellation outcomes. It prevents a misleading conceptual flow in which image dimensions alone choose tiled classification or a cancellation request immediately terminates a model operation.

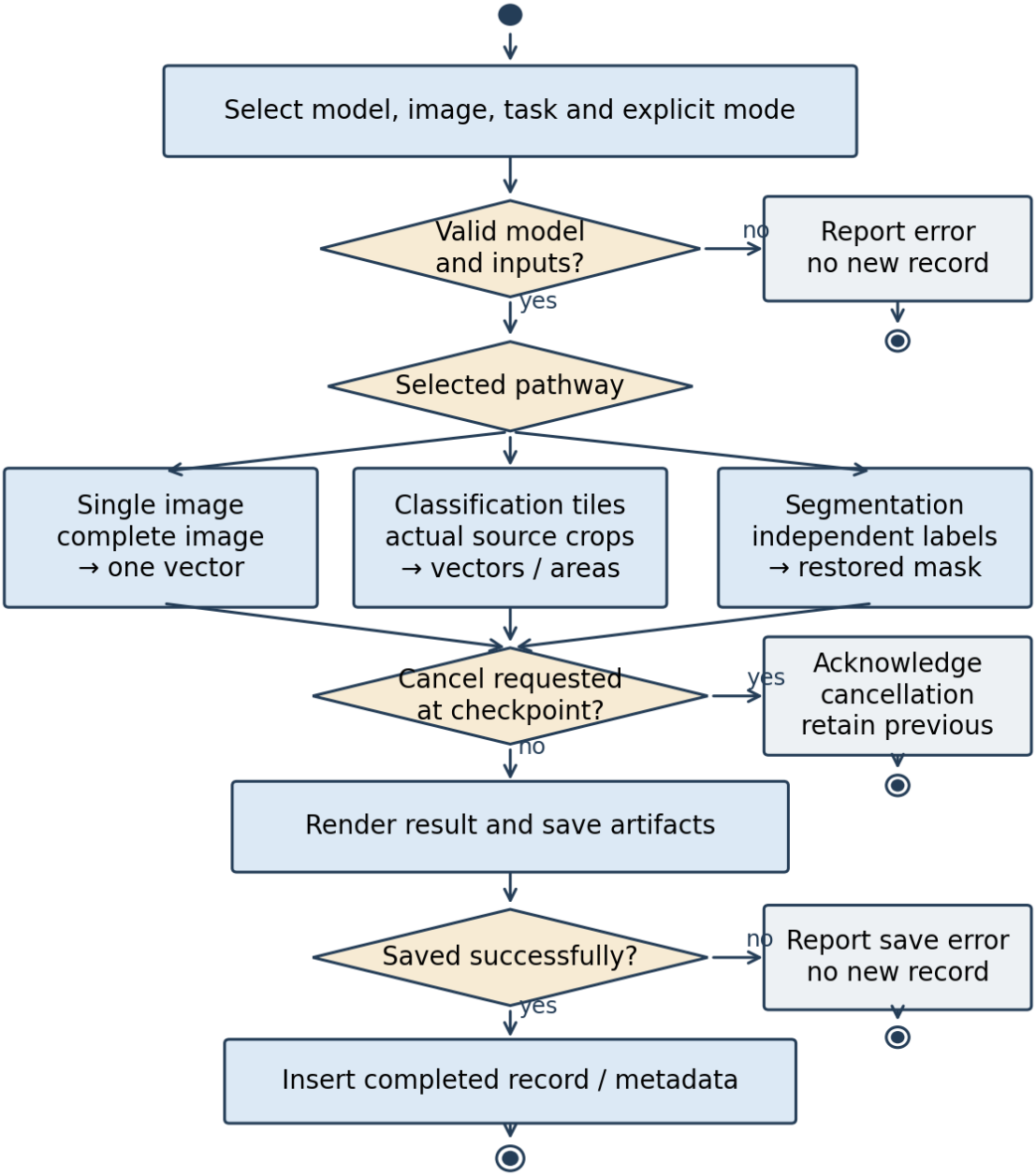


Figure 3.6. Activity diagram from parameter selection to completed persistence, with safe cancellation and error exits. Source: author, based on the implemented application.

3.6 Testing Methodology

3.6.1 Development Testing

Development verification combines component checks with cross-module tests using pytest and pytest-qt. The tests exercise model contracts, preprocessing, inference routing, persistence and user interaction at their observable interfaces.

Controlled cases use known in-memory networks, synthetic tensors and raster images, temporary SQLite databases and isolated QSettings stores. Real-file availability checks remain separate, so successful fixtures cannot substitute for the presence or validation of production model files.

The latest Windows test record uses Python 3.10.22, pytest 9.1.1, pluggy 1.6.0, PyQt5 5.15.11, Qt runtime/compiled version 5.15.2, anyio 4.15.1 and pytest-qt 4.5.0. PyTorch 2.1.2+cpu reported CUDA unavailable and 24 CPU threads. Table 4.4 gives the host hardware and separates its installed GPU from the actual computation device.

Assertions check external behaviour: crop coordinates and input values, area conservation, mode routing, probability order, saved metadata, rerun isolation, cancellation outcomes and accessible controls. GUI input tests use mouse/keyboard events, not only programmatic setChecked calls. Presentation tests check that Fit, zoom and grid changes do not alter analysis data. A screenshot can support inspection but does not replace these behavioural assertions.

3.6.2 Module Testing and Fallback Handling

The loader is tested against supported representations, invalid metadata, missing/empty files, wrong shapes and non-finite logits. Preprocessor tests verify RGB conversion, profile values and resize backends. Tile tests verify unique crops, right-exclusive rectangles, partial-edge areas and agreement of a direct crop with the same preprocessing pathway. Parser tests check softmax/class mapping and mask restoration.

Database and export tests use temporary artifacts to check CRUD operations, date bounds, report metadata and error propagation. Rerun tests change current selections/defaults to ensure they cannot overwrite recorded task or model

parameters. Cancellation tests cover requests before load, at pipeline checkpoints, on the final tile and during GUI cleanup, including the rejection of a late result.

Fallback is intentionally constrained. Missing weights, missing cached UPerNet configuration and incompatible checkpoints produce explicit errors. An unsupported architecture is not replaced by a random network. A missing recorded model is not replaced with the current default. The software can remain available for other operations after an error, but “graceful handling” does not mean silently obtaining a prediction from a different model.

3.6.3 Manual System Testing

Manual testing supplements automated checks on the actual Windows installation. The evidence checklist records software commit, Python/dependency versions, display scale, available screen geometry, model checksum if available, source dimensions and selected parameters. Each case records expected behaviour, actual behaviour, pass/fail, screenshot/log identifier and tester/date. Untested cases remain marked pending.

The acceptance cases include both classification modes, segmentation, cancellation, source comparison, Fit/1:1, narrow/wide transitions, maximise/restore, recorded rerun, deletion, export, calendar interaction and task-specific defaults. A formal case receives a Pass status only when its expected outcome has a completed manual execution record. Section 4.3.3 separates these records from the completed native Windows automated suite.

3.6.4 User Acceptance Testing (UAT)

The planned pilot retains the CP1 target of approximately 8–10 participants, subject to recruitment, consent and institutional requirements. Participants should include intended educational/research users with varied programming and pathology experience. Recruitment is purposive/convenience sampling rather than a representative clinical sample; results should be reported accordingly.

Participants receive the same orientation and authorised sample files, then attempt a fixed task sequence: load/classify an image; select tiled analysis and inspect a tile; explain tile coverage versus confidence; compare aligned source/result regions; inspect segmentation; locate and restore history; rerun after changing current

defaults; export; cancel; and configure/restore a task-specific model default. Assisted completion is recorded separately from independent completion. Cancellation is judged against the intended safe-stop outcome, not the speed of killing a thread.

After the tasks, participants complete the unmodified ten SUS items, separate feature ratings and open-ended questions. Completion time, assistance, errors and interpretation responses are recorded. There are no actual participant responses at the time of writing. Appendix B contains the consent briefing, tasks, questionnaire and recording rules; the accompanying workbook is a data-entry/scoring template, not a populated dataset.

For participant i with item responses x_{ij} in 1–5, the SUS score is:

$$SUS_i = 2.5 \times [\Sigma(\text{odd } j)(x_{ij}-1) + \Sigma(\text{even } j)(5-x_{ij})]. \quad (3.4)$$

Score each participant only when all ten responses are valid integers from 1 to 5, then calculate the sample mean, sample standard deviation, median and range; the IQR may also be reported. Missing items remain unscored. Independent and assisted task completion are separate outcomes, and feature ratings exclude Not used with their own denominators.

3.6.5 Testing Summary

Three evidence streams are kept separate. Automated development/integration tests establish repeatable technical behaviour under controlled fixtures. Manual Windows testing establishes behaviour on the intended operating system, display configurations and real files. UAT evaluates observed task performance and user perceptions. Independent model evaluation is a fourth, distinct activity needed for predictive claims; no amount of GUI testing replaces it.

Automated results are labelled by environment, date and evaluated source state. On 19 October 2026, the final native Windows invocation completed all 370 tests with no failures, errors or skips. A preceding 131-test regression subset also passed; its count is not added to the full-suite total. Table 4.3 retains earlier invocations separately.

3.6.6 Requirements Traceability Matrix

The traceability matrix connects objectives and requirements to source modules and named tests. Its statuses distinguish completed Windows and Linux automated checks, the recorded classifier evaluation and outstanding manual acceptance, UAT and model studies.

Table 3.6. Requirements traceability matrix

Requirement / objective	Implementation	Principal executable evidence	Status and boundary
F01 / O3	InferencePanel; image opening in InferenceEngine	test_inference_panel.py; test_inference_engine.py	Windows full-suite image-loading and input-error cases passed; representative real-image manual tasks pending.
F02 / O2	ModelLoader; ModelValidationThread	test_model_loader.py	Windows loader/probe and built-in-path cases passed; real-classifier validation recorded. Arbitrary-model semantic compatibility is outside this check.
F03 / O3	Preprocessor; OutputParser	test_preprocessor.py; test_output_parser.py; test_inference_engine.py	Windows software cases passed; default-Pillow patch accuracy 90.68%, macro-F1 88.59% on native 224 × 224 inputs (Section 4.2.6).
F04–F05 / O3–O4	ClassificationTileSpec; predict_classification_tiles	test_classification_tiles.py; test_classification_modes_display.py	Windows crop, mode, legend and actual-area fixture cases passed; tiled-map predictive evaluation pending.
F06 / O3–O4	large_image_segmentation; SegmentationResult	test_large_image_segmentation.py; test_segmentation_results_layout.py	Windows synthetic segmentation geometry/statistics and display cases passed; real-weight demonstration and ground-truth evaluation pending.
F07 / O4	ImageComparison; ZoomableImageView	test_linked_image_comparison.py; test_screen_fit_and_tile_grid.py	Windows coordinate, linked-view, Fit/zoom/grid and compact-viewport cases passed; formal real-image manual acceptance pending.
F08 / O6	DatabaseManager; MainWindow artifact saving	test_database_manager.py; test_main_window_inference.py	Windows persistence and artifact cases passed with temporary test records; real-image manual workflow evidence pending.

Requirement / objective	Implementation	Principal executable evidence	Status and boundary
F09 / O7	HistoryPanel search/local-date queries	test_history_panel.py; test_result_controls_and_calendar.py	Windows query, calendar and history-interaction cases passed; user task outcomes await UAT.
F10–F11 / O7	Stored restoration and recorded rerun	test_main_window_rerun.py; test_classification_modes_display.py	Windows restore and recorded-model/mode/parameter rerun cases passed; user-efficiency assessment awaits UAT.
F12 / O5	ExportManager and export coordinator	test_export_manager.py; test_main_window_export.py	Windows file/report and recorded-metadata cases passed; final real-image report screenshots pending.
F13 / O1	Cooperative cancel and close lifecycle	test_cancellation_and_model_defaults.py; test_worker_lifetime.py	Windows cooperative-stop, late-result and destruction/callback lifecycle cases passed; real-model manual cancellation record pending.
F14–F15 / O1–O3	ModelSettingsPage; SettingsDialog; QSettings	test_cancellation_and_model_defaults.py; test_preferences_interaction.py	Windows defaults, validation, preferences and history-isolation cases passed.
F16 / O6–O7	Confirmed record/artifact deletion	test_main_window_delete.py; test_database_manager.py	Windows app-owned artifact cleanup and source-preservation cases passed.
N01, N04 / O1, O4	Worker signals; logical screen bounds; scroll layouts	test_window_lifecycle_and_theme.py; test_settings_polish_and_resize.py	Windows window-state, restore, small-viewport and layout cases passed; multi-monitor/assistive-technology manual acceptance pending.
N02, N06 / O2, O6–O7	Metadata validation and explicit errors	test_main_window_rerun.py; test_cancellation_and_model_defaults.py	Windows missing-file/error handling and recorded-parameter cases passed; immutable provenance is not implemented.
N03 / O1	Local filesystem/SQLite; no inference service	Source inspection of core/data and utils/paths.py	Local execution established by design and recorded source startup; security audit not performed.

Requirement / objective	Implementation	Principal executable evidence	Status and boundary
N05, N07–N08 / O1	Runtime/ deployment/ regression process	Recorded pytest logs; planned benchmark/package protocol	Windows full suite: 370 passed, 0 failed/errors/skipped; repeated benchmark, clean package and coverage measurement pending.

Source: named project tests, the environment-labelled Linux records and the completed native Windows full-suite log/XML of 19 October 2026, together with the classifier evaluation.

Appendix C indexes the supporting records. Fixture tests, model evaluation, native observations and UAT each address different acceptance questions, so the matrix records their scope separately.

3.7 Risk Management and Mitigation Strategies

The CP1 risks concerning model compatibility, dependencies, packaging, CPU performance, recruitment and data reliability remain relevant. CP2 adds scale ambiguity and misleading spatial statistics because explicit tiled classification introduces sampling choices that a user might mistake for validated tissue measurements. Table 3.7 distinguishes implemented controls from future verification.

Table 3.7. Risk register and residual mitigations

Risk	Consequence	Existing control	Remaining action
R1: model/preprocessing mismatch	Structurally valid but wrong input semantics	Metadata checks, explicit profiles/backends and ordered labels	Confirm physical scale, resampling beyond native-sized patches and dataset grouping; retain the selected checkpoint checksum.
R2: missing weights/cache/package dependencies	Default model cannot run offline	Clear errors; local-only UPerNet configuration request	Verify segmentation resources and permitted caches in a clean Windows package
R3: class-map/coverage misunderstanding	Tile estimates mistaken for pixel boundaries or diagnosis	Separate modes, palettes, legends and area labels	UAT interpretation task and training-scale confirmation

Risk	Consequence	Existing control	Remaining action
R4: CPU/resource limits	Long runs or unresponsive experience	Worker execution, progress, cooperative cancel	Fixed-workload time/RAM benchmark and memory-boundary study
R5: window/DPI regressions	Hidden actions, clipped text or misaligned views	Logical-coordinate bounds, scroll areas, repeated resize tests	Native Windows docking/multi-monitor verification
R6: history/artifact loss	Incomplete restore or rerun	Transactions, file checks and app-owned deletion bounds	Backup/recovery policy and stronger artifact transaction design
R7: mutable paths/dependency drift	Rerun differs from the original	Recorded task/model path/parameters and revision logs	Per-run hashes, environment manifests and migration/version policy
R8: malicious or untrusted model	Code execution during deserialisation	Trusted-source workflow; weights-only load attempt	Signed/checksummed releases and restricted supported formats
R9: limited UAT recruitment	Uncertain usability/generalisation	Prepared protocol, questionnaire and task logs	Recruit intended users and report actual sample limitations
R10: overstatement of evidence	Unsupported accuracy or clinical claims	Separate software, model, platform and user evidence	Complete UAT, formal real-image Windows acceptance and segmentation evaluation; retain each executed result with its environment and source state.

Source: author's implementation risk assessment.

The register does not imply that every listed mitigation is implemented. In particular, backups, signed models, immutable provenance and clean-machine packaging are future actions. Risk review should be repeated when a model, dependency, display configuration or artifact schema changes.

3.8 Summary

The methodology combines independent task contracts, source-aware classification geometry, spatial inspection, recorded rerun and cooperative cancellation. Evaluation combines controlled software tests completed on native Windows, the

classification training/test records and source-startup observations; UAT and formal manual acceptance follow separate protocols. Chapter 4 presents these results and their implications.

Chapter 4: Results and Discussion

4.1 Introduction

This chapter reports the implemented system and the evaluated source state identified in Appendix C. Section 4.2 presents implemented workflows and a distinct classifier evaluation; Section 4.3 reports software tests and native observations; Section 4.4 reserves the UAT results. Environment and date labels identify each completed or interrupted invocation.

Results are organised into implementation behaviour, automated verification, Windows runtime observations, classification performance and the planned UAT. The evidence boundaries, including segmentation evaluation, performance measurement and deployment, are consolidated in Section 4.5.3.

4.2 System Implementation Results

4.2.1 Implemented workflow and scope

ZY-Path presents a local three-region interface with input/model controls, central inference/history views and export/session actions. The implemented features correspond to the functional requirements in Chapter 3: supported image/model selection, model validation, explicit classification modes, segmentation, spatial inspection, saved records, retrieval/rerun, cancellation, defaults, deletion and export.

The Windows session on 19 October 2026 launched the application from its virtual environment and recorded successful loading and validation of crc_dn21_seed42_best.pth as a nine-class DenseNet with output shape (1, 9). This establishes source-based startup and classifier integration for that session. The observed workflow and window states are detailed in Section 4.3.3.

EVIDENCE TO INSERT — Final Windows application overview

Capture the current committed application in a normal window, with readable input, result and session/export panels. Show commit/version evidence separately; redact personal paths. Use a real authorised input and identify the selected task/mode.

Figure 4.1. Final Windows application overview (evidence pending; replace this reserved area with the actual authorised capture).

4.2.2 Classification results and spatial inspection

Single-image classification retains the original whole-image label, confidence and probability-bar chart. Tiled analysis produces a source-sized class-colour map, default overlay, nine-class coverage cards and detailed per-tile labels/confidences. The source tile width and height are editable, and the main panel explicitly shows Single image or Tiled analysis before running. Class order and palette remain independent of segmentation.

Figure 4.2 is a current Qt result-view capture using a deterministic synthetic fixture. It demonstrates the grid, paired views and statistics placement. The fixture deliberately contains 49 tiles with 47 STR, one NORM and one MUC assignments. These assignments are prescribed test data, not model predictions on the coloured source pattern and not evidence of classifier accuracy.

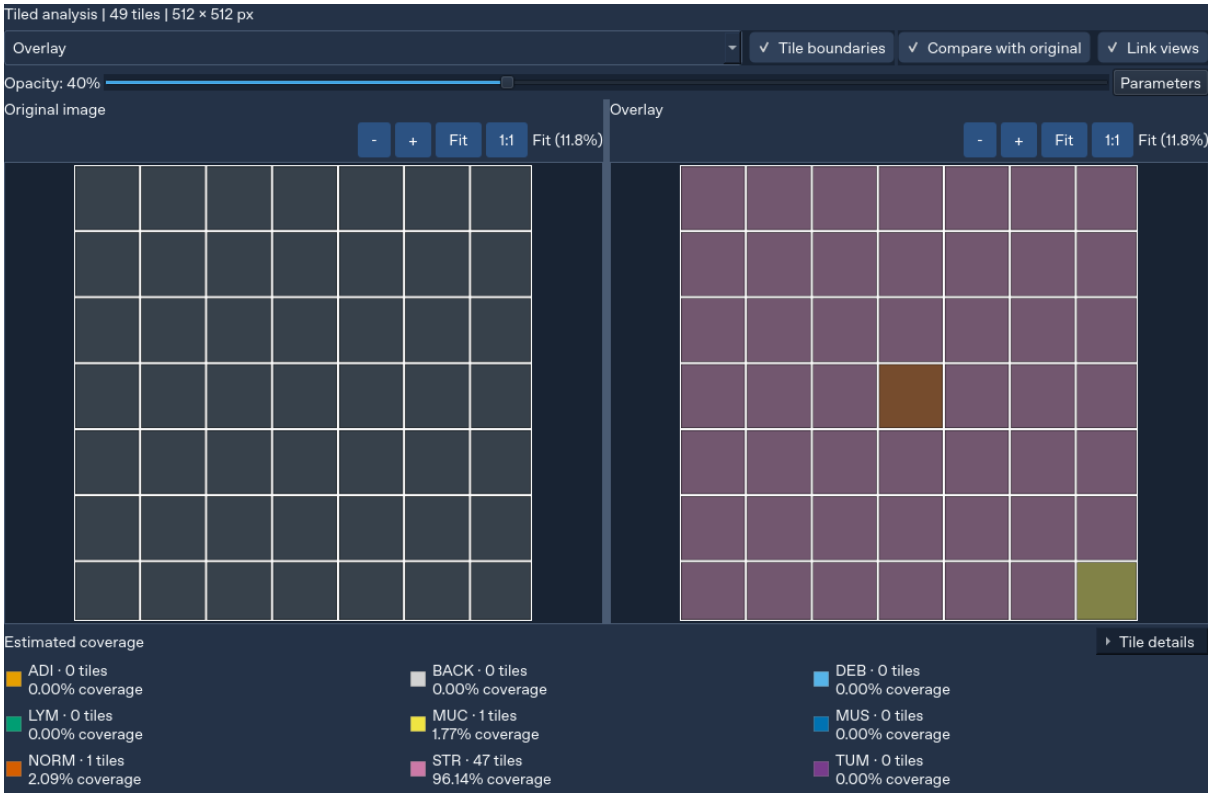


Figure 4.2. Tiled classification interface with source/result comparison and revised result toggles. Source: author's Linux/offscreen-Qt development capture; synthetic fixture, not a model result.

EVIDENCE TO INSERT — Real-weight single-image and tiled classification evidence

Provide two labelled panels from the same authorised raster image: Single image with probability chart, and Tiled analysis with mode summary, tile dimensions, nine-class legend and per-tile detail. Include image size, resize backend and model identity. Do not claim identical predictions between whole-image and crop-based modes.

Figure 4.3. Real-weight single-image and tiled classification evidence (evidence pending; replace this reserved area with the actual authorised capture).

The 3552 × 3536 geometry example produces 49 crops at a 512 × 512 setting, with smaller right and bottom edges. Controlled tests verify unique source crops, RGB/raw255 preparation, class order, edge handling and area conservation. The tissue accuracy of a tiled map requires a separately defined ground-truth and scale protocol; the patch-classification results in Section 4.2.6 do not measure that map.

4.2.3 Segmentation results

The segmentation result viewer now follows the same inspection pattern: an initial overlay without duplicate source preview, a pure segmentation map option, opacity control, optional original comparison and linked navigation. Its legend and restored-mask pixel counts use segmentation labels. Classification preferences do not alter its source tile/stride/input defaults.

The reference notebook establishes the segmentation architecture, preprocessing, labels and probability-fusion procedure. Controlled software tests cover restored dimensions, class counts and blending behaviour. Segmentation predictive performance is treated separately: annotated-mask metrics were not available for this model, as discussed in Section 4.5.3.

EVIDENCE TO INSERT — Windows segmentation result and aligned comparison

Capture Overlay and Compare with original using the actual segmentation model. Include the full legend, pixel-coverage label, model/session identity and matched zoom percentages. Record source dimensions and segmentation tile/stride/input metadata.

Figure 4.4. Windows segmentation result and aligned comparison (evidence pending; replace this reserved area with the actual authorised capture).

4.2.4 History, rerun and export

History stores completed records and supports keyword/date filtering, restoration and a separate rerun action. A tiled-classification row reports tile count and the largest estimated class coverage; its Confidence cell remains “-” rather than implying

a whole-image confidence. Detailed per-tile confidence is available when the result is restored. The internal aggregate remains available with its explicitly different PDF terminology.

Rerun tests demonstrate that the recorded model/task/mode and analysis settings take precedence over current preferences. Restoring a record does not itself create a new prediction. The export workflow uses the displayed completed record's metadata and can retain task-specific maps/JSON together with the presentation. This prevents a newly selected model from being incorrectly attached to an older visible result.

EVIDENCE TO INSERT — History and recorded-parameter rerun

Show a real Single image row and Tiled analysis row, then restore the tiled record after changing current preferences. Capture its recorded model, mode, tile dimensions and backend. Show a separate new record after an intentional rerun.

Figure 4.5. History and recorded-parameter rerun (evidence pending; replace this reserved area with the actual authorised capture).

EVIDENCE TO INSERT — Exported report and companion artifacts

Capture an opened PDF report plus the exported files in a folder. Ensure the report identifies the actual task/model/mode; show classification tile geometry/estimated coverage or segmentation parameters as appropriate. Remove identifying source paths before submission.

Figure 4.6. Exported report and companion artifacts (evidence pending; replace this reserved area with the actual authorised capture).

4.2.5 Settings, cancellation and interface polish

Settings includes Appearance, Preferences, Models, Guide & Support, System Information and About. Model defaults are independent per task and require validation. Preferences applies the classification mode immediately while retaining a separate startup default; closed dropdowns scroll the page instead of changing selections accidentally. Guide navigation remains visible and follows the current reading section, while the system-information copy confirmation expires rather than becoming permanent content.



Figure 4.7. Implemented Guide & Support page with persistent Contents navigation and numbered guidance. Source: author's Linux/offscreen-Qt development capture; this is interface evidence.

The English calendar in Figure 4.8 was captured after setting the default locale to Chinese in a controlled run. The corresponding test opens each popup, checks English month/day labels, selects a date and verifies the unchanged date value and query signal. This is stronger evidence than merely setting a locale property without opening the widget.



Figure 4.8. English history-calendar popup with a simulated Chinese default locale. Source: author's Linux/offscreen-Qt development capture; the selected date is test input.

Cancel inference is available during work. Tests cover cancellation at multiple checkpoints and preservation of the previous completed result. The interface waits for the current operation and worker cleanup; it does not advertise instantaneous termination. A final real-model cancellation recording remains useful because the duration of an individual model operation depends on the actual weights, workload and hardware.

EVIDENCE TO INSERT — Cancellation and per-task model defaults on Windows

Capture a running tile operation, the Cancelling state and the unchanged previous completed record after acknowledgement. Separately show a validated local default and restoring that task's built-in default. This is a multi-panel evidence figure, not a claim of instant interruption.

Figure 4.9. Cancellation and per-task model defaults on Windows (evidence pending; replace this reserved area with the actual authorised capture).

4.2.6 Classification training and test results

The formal experiment uses the seed-42 NCT-CRC-HE-100K manifests with 80,002 training and 19,998 validation images. Their hashes match config.json, and relative paths do not overlap. The training log identifies an NVIDIA L4. This training environment is separate from the CPU-only Windows application session.

The 30-epoch metrics record selects epoch 26 by minimum validation loss (0.0275519), with validation accuracy 99.09% and macro-F1 99.09%. The directory name crc-dn21-seed42-10ep and config.json epochs=10 describe the initial run configuration; continuation logs and metrics.csv establish the completed 30-epoch history. Figure 4.10 illustrates the epoch-to-epoch variation in validation loss and accuracy.

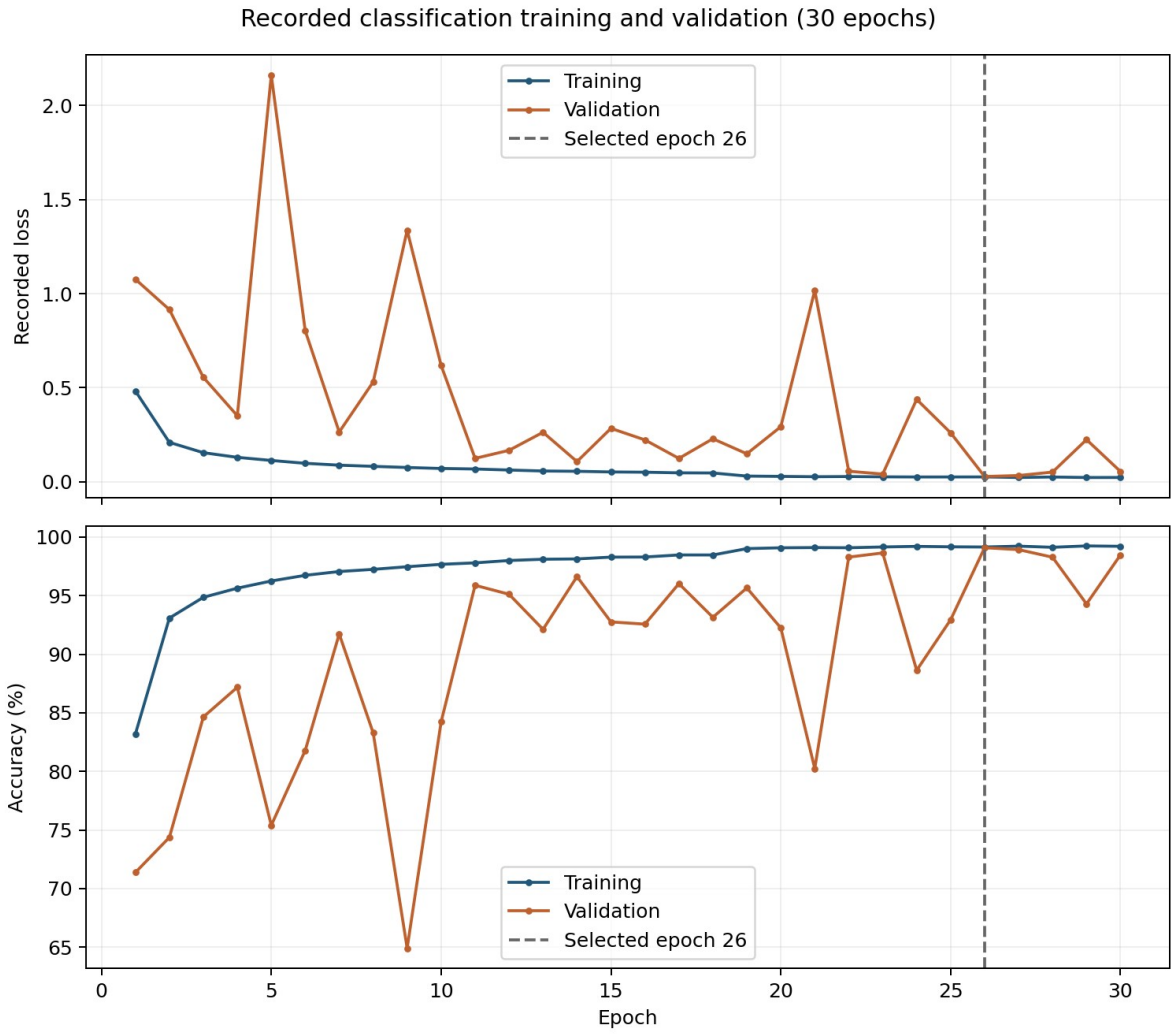


Figure 4.10. Classification training and validation over 30 recorded epochs, with the selected epoch 26 marked. Source: author's plot of experiments/crc-dn21-seed42-10ep/metrics.csv.

Table 4.1. Classification evaluation summary

Measure	Recorded / recomputed result	Scope
Test collection	CRC-VAL-HE-7K; 7,180 images	Separate from the internal training/validation manifests
Correct / incorrect	6,511 / 669	Both runs: independently recomputed from their prediction CSVs
Test accuracy	90.682451%	Original and default-Pillow runs: 6,511 / 7,180
Test macro-F1	0.885879 (88.59%)	Both runs; unweighted mean of nine class F1 values
Internal validation	99.09% accuracy; 99.09% macro-F1	Selected epoch 26; not the external test result

Measure	Recorded / recomputed result	Scope
Checkpoint identity	Epoch-26 best_model.pth	SHA-256 matches summary.json; renamed for application use
Default application input	7,180 native 224 × 224 RGB patches	Pillow; float32 raw255; CPU batch 1
Prediction agreement	7,180 / 7,180 reference labels	Max probability difference: 0.002732

Source: the classification experiment's metrics.csv and test-7k outputs, together with the Windows default-Pillow evaluation of 19 October 2026. Per-image predictions from both runs reproduce the displayed aggregate results.

Table 4.2. Class-wise classification performance on 7,180 test images

Class (support)	Precision (%)	Recall (%)	F1 (%)
ADI (1,338)	97.43	84.90	90.73
BACK (847)	98.83	100.00	99.41
DEB (339)	79.76	98.82	88.27
LYM (634)	95.17	99.37	97.22
MUC (1,035)	96.59	93.14	94.84
MUS (592)	61.24	73.14	66.67
NORM (741)	90.21	94.47	92.29
STR (421)	79.14	65.80	71.85
TUM (1,233)	95.58	96.43	96.00

Source: class-wise reports and per-image predictions from the original evaluation and Windows default-Pillow rerun. Both runs give the same class counts and metrics; support is the number of true-class images. Displayed values are rounded.

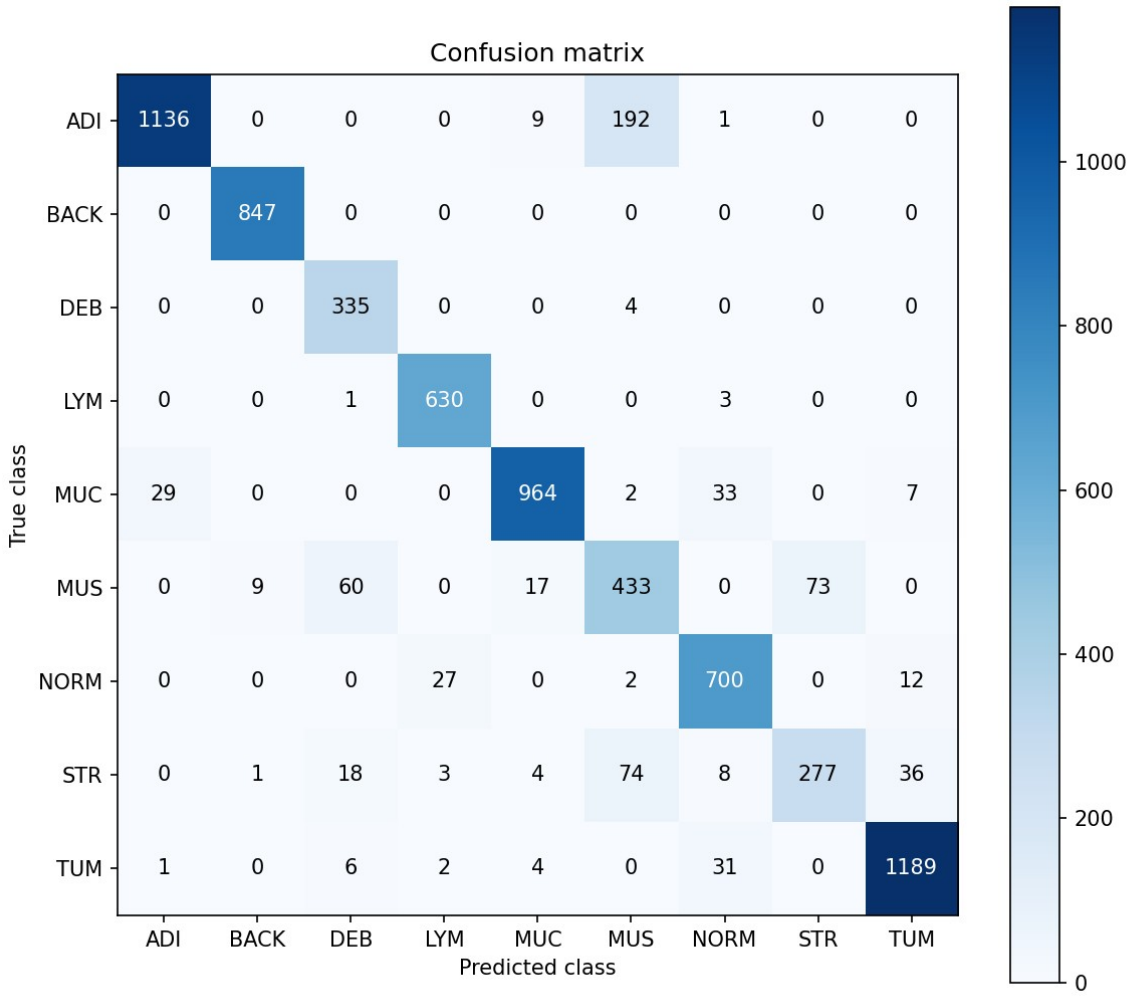


Figure 4.11. Nine-class confusion matrix for CRC-VAL-HE-7K. Source: the classification experiment's saved matrix; the Windows default-Pillow predictions independently reproduce every count. Rows are true labels and columns predicted labels.

Performance varies substantially by category. BACK has F1 99.41%, while MUS and STR have F1 66.67% and 71.85%. The largest off-diagonal count is ADI predicted as MUS (192 images); MUS/STR confusion is bidirectional (73 MUS→STR and 74 STR→MUS). DEB recall is 98.82% but precision is 79.76%, showing that high sensitivity for one class can coexist with false positives. These differences justify presenting class-wise results alongside accuracy.

The selected-epoch validation accuracy exceeds test accuracy by approximately 8.41 percentage points. Internal validation draws from the same 100,000-image pool as training, whereas the 7,180-image test is a separate collection. Class proportions also differ: ADI represents 10.41% of the 100K pool versus 18.64% of the test set, DEB 11.51% versus 4.72%, and MUS 13.54% versus 8.25%. These observed

differences do not identify a single cause of the gap. Both validation and original test scripts use deterministic Resize, ToTensorV2 and float32 raw255; the Windows application-path rerun produces the same test labels and does not explain away that gap.

Training used the NCT-CRC-HE-100K.zip archive, identified as the standard normalised release rather than the separately named non-normalised alternative. The dataset authors' repository documents that normalised 100K release and the availability of the alternative (Kather, n.d.). Stain normalisation of source images is distinct from numerical raw255/ImageNet tensor profiles. The test archive's stain processing and acquisition matching were not independently characterised, so the score gap is not attributed to a proven stain mismatch.

Both the class-wise results and the lower macro-F1 show that a global accuracy number alone is insufficient. Patient/slide grouping, archive identity, stain characteristics and source resolution should be recorded in a follow-up evaluation. The current image-path checks establish disjoint file lists, not patient-level independence.

Both prediction CSVs contain 7,180 unique image paths with matching true labels, valid class argmax, maximum softmax score and probability sums. Recalculation gives 6,511 correct predictions, 90.682451% accuracy and 88.587944% macro-F1 for each run. Every class-wise metric and confusion-matrix entry agrees. The deployed classifier has the same checkpoint checksum as the original evaluated model; Appendix E records the execution configurations.

On 19 October 2026, I evaluated the labelled patches on Windows through the project's ModelLoader, Preprocessor and OutputParser using the unchanged Pillow default, RGB float32 raw255, a 224 × 224 input and CPU batch size 1. All source patches were already 224 × 224. Input diagnostics against the installed OpenCV decoder and INTER_LINEAR produced zero tensor differences; the reference comparison retained all 7,180 predicted labels. Probability vectors were not identical: the maximum absolute probability difference was 0.002732. This comparison confirms native-sized patch predictions across the recorded runs, without establishing identical numerical output or resize behaviour at other source sizes.

The application-default Single image core therefore has a measured result on this labelled patch collection. Arbitrary 512-pixel source crops, tiled maps and physical-scale changes require their own ground-truth and scale design. Patient/slide independence and these generalisation limits are discussed in Section 4.5.3.

4.3 Functional Testing Results

4.3.1 Recorded automated results

The final Windows invocation on 19 October 2026 completed the entire 370-test suite with 370 passed, 0 failed, 0 errors and 0 skipped, with 15 warnings in 194.10 seconds (exit 0). It used the native windows Qt platform, Python 3.10.22, PyQt5 5.15.11 and Qt 5.15.2. The evaluated source includes the lifecycle and layout repairs; Appendix C records its baseline and per-file identity.

Table 4.3. Automated verification records by environment and evaluated source state

Run / environment	Passed	Failed	Collection errors	Interpretation
Windows, 19 Oct; final full run	370	0	0	0 skipped; native Qt; 15 warnings; 194.10 s; exit 0. Baseline plus reviewed repairs.
Linux, 19 Oct; final full run	369	1	0	370 collected; 14 warnings; 125.70 s; exit 1. Real segmentation weight unavailable.
Linux, 19 Oct; before repairs	358	1	0	359 collected; 14 warnings; 104.68 s; exit 1. Real segmentation weight unavailable.
Windows, 19 Oct; 15:58 run	Not final	At least 2	0 reported	359 collected; runtime termination; exit 0xC0000409. No final summary.
Windows, 19 Oct 2026; first run	Not run	Not run	1	356 collected; collection interrupted; 13 warnings; 24.76 s; exit 2.
Linux, 17 Oct 2026	355	1	1	Continuation after collection errors; 14 warnings; 110.70 s.

Source: dated pytest logs, XML summaries, exit codes and source-state records. The final Windows run began at 22:26 on 19 October 2026 (UTC+08:00); earlier invocations remain separate. Related subset counts are not additive.

Before the full invocation, the related Windows subset passed 131 tests in 119.90 seconds with 15 warnings and exit 0. The full command selected no modules or individual cases and used no collection-error continuation. XML confirms 370 distinct passing cases and no skips. The 15 warnings concern deprecated Matplotlib/Pyparsing, PyTorch TypedStorage and Hugging Face download interfaces.

Table 4.4. Windows test host and actual CPU runtime on 19 October 2026

Category	Recorded specification	Interpretation
Operating system	Microsoft Windows 11 Home Chinese Edition; 10.0.22621, Build 22621; 64-bit	Windows Python/Qt environment; native acceptance assessed separately
CPU	13th Gen Intel Core i9-13900HX; 24 cores; 32 logical processors	PyTorch CPU threads: 24
Memory	15.81 GiB physical RAM	Physical system memory
Installed GPU	NVIDIA GeForce RTX 4060 Laptop GPU; driver 32.0.15.6094	Installed hardware only; not used for these runs
Actual compute	PyTorch 2.1.2+cpu; CUDA available=False	CPU execution; no GPU acceleration claim
Python/test stack	Python 3.10.22; pytest 9.1.1; pluggy 1.6.0; anyio 4.15.1; pytest-qt 4.5.0	Versions from the Windows test record
GUI stack	PyQt5 5.15.11; Qt runtime 5.15.2; Qt compiled 5.15.2	Startup: DPR 2.0; automation: DPR 1.0, logical DPI 168. Each geometry record uses Qt logical coordinates.

Source: Windows test/environment record dated 19 October 2026; full revision and evidence-file index in Appendix C. Installed GPU specifications are separated from the actual CPU runtime.

The final Linux/offscreen invocation after the same repairs completed with 369 passed, 1 failed and no collection errors, with 14 warnings in 125.70 seconds. Its remaining failure checks availability of the real segmentation weight, which was absent from that installation. Earlier 17 and 19 October results in Table 4.3 document earlier source/resource states rather than the final Windows outcome.

Windows complete automated suite — 19 October 2026

```
platform win32 -- Python 3.10.22, pytest-9.1.1, pluggy-1.6.0

PyQt5 5.15.11 -- Qt runtime 5.15.2 -- Qt compiled 5.15.2

collected 370 items

[... intervening test output omitted ...]

370 passed, 15 warnings in 194.10s (0:03:14)
```

Selected recorded lines; omissions marked. Source: full pytest log.

Figure 4.12. Completed Windows pytest report, 19 October 2026. Source: the recorded full-suite log; selected text is reproduced with omissions marked.

The final Windows suite passed the built-in-model availability assertion and the previously affected history, rerun, worker-lifetime and responsive-layout cases. The separate default-Pillow classifier evaluation in Section 4.2.6 supplies predictive measurements; passing software tests and file-availability checks do not measure segmentation accuracy.

Earlier Windows invocations stopped first on the missing training-script prerequisite and later on a running-thread destruction fault. Restoring the genuine script and repairing thread lifetime, queued validation receivers and panel size policies resolved those observed failures in the final full invocation. Table 4.5 summarises the implementation causes and regression evidence.

4.3.2 Challenges, resolutions and evidence

Table 4.5. Implementation challenges and verified resolutions

Challenge	Actual cause or risk	Resolution	Evidence and limit
Missing/incorrect classifier assumptions	CP1 ResNet/ImageNet description disagreed with implemented DenseNet/raw255 contract	Use the actual nine-class DenseNet/raw255 metadata, training records and class order	Windows default-Pillow patch accuracy 90.68%, macro-F1 88.59%; agreement measured on native 224 × 224 images.

Challenge	Actual cause or risk	Resolution	Evidence and limit
Misleading tiled history confidence	Aggregate tile output could appear to be a whole-image label	Tile summary and estimated coverage; separate aggregate PDF terminology	History/export semantic tests; no prediction values changed
Fit grid lines disappearing	Fine rasterised grid could vanish when the large image was reduced	Paint vector grid after the image using the same transform	Source-coordinate/grid tests across Fit, zoom and resize
Comparison stayed stacked after widening	A side legend could shrink the comparison below its breakpoint	Keep segmentation legend below paired images	Repeated main-window narrow/wide regression for both tasks
Oversized/clipped windows and settings	Child minimum sizes and wrapped-text sizing constrained the layout	Logical screen bounds, inner scrolling, measured legend/card sizes and pinned action space.	Windows small-screen/font, maximise/restore and compact-viewport cases passed; formal manual task evidence pending.
Tight focus box and unreliable label/padding clicks	Global focus border and native checkbox hit region differed	Stable rounded surface; full-rectangle hit area; keyboard-only focus cue	Mouse edge/text/padding, disabled and Tab/Space tests in both themes
Accidental preference changes by wheel	Closed combo boxes consumed scroll as selection input	Explicit selection; wheel forwarded to page scrolling	Popup/keyboard/wheel integration tests
Unclear cancellation/default ownership	Late worker results or current defaults could affect a record	Cooperative stop/late-result rejection; per-task defaults and recorded rerun	Cancellation/default/rerun tests with temporary artifacts
Calendar followed local language	Date popup inherited locale-dependent text	Explicit English locale for editors and calendars	Actual popup/day selection under Chinese default locale
Worker destroyed during model loading	Forced parent deletion bypassed deferred close while a QThread was still running	Wait for worker completion before destroying child threads; retain asynchronous normal close	Native fatal message identified cause; two worker-destruction regressions and final Windows suite passed.
Validation callback accessed deleted controls	Queued lambda outlived its owning window or settings page	Use QObject-bound receivers disconnected with their owner	Six successful/error callback lifecycle cases passed for built-in, custom and task-default validation.

Challenge	Actual cause or risk	Resolution	Evidence and limit
Run/Re-run clipped in short viewports	Right-side inner scroll-area height hint enlarged the full three-panel splitter	Allow vertical shrinking of scrolling content; retain pinned action space	16/30px-scrollbar compact-body regressions and the original 480 × 360 Windows case passed.

Source: implementation changes and their named regression cases, including the completed native Windows suite of 19 October 2026.

The corrected cases retain their original behavioural checks; the suite additionally includes eleven lifecycle and viewport regression cases introduced during refinement. The Windows maximisation check uses the client area and Qt's actual native frame margins, while normal/restored full-frame containment remains required. Classification performance is reported separately in Section 4.2.6.

4.3.3 Windows startup and window-state observations

Windows source execution recorded successful loading and validation of the built-in classifier at approximately 14:23 on 19 October 2026. Loading completed from the checkpoint; validation identified DenseNet, classification, nine classes and output shape (1, 9), and the main window accepted the model. The later default-Pillow evaluation completed 7,180 real-patch predictions (Section 4.2.6).

The source-startup session reported devicePixelRatio=2.0 and availableGeometry=[0, 0, 1280, 758] in Qt logical coordinates, with normal, maximised and minimised state entries. The automated Windows application reported DPR 1.0, logical DPI 168 and availableGeometry=[0, 0, 2560, 1516]. Each record is interpreted within its own Qt application; these logical values are not substituted for a measured physical monitor resolution.

Native Windows automated checks now cover maximise/restore, saved-geometry recovery, repeated narrow/wide comparison, grid alignment and short viewports with accessible Run/Re-run actions. These completed checks resolve the earlier engineering regressions. Appendix D still reserves formal manual outcomes for authorised real-image workflows, real-model cancellation, screenshot verification and the intended display configurations.

4.3.4 Runtime and deployment verification status

The hardware and CPU runtime are documented in Table 4.4, but no repeated stage-level measurements were recorded for loading, preprocessing, forward inference, reconstruction/statistics, GUI display, saving or export. Cold and warm runs have not been separately summarised. Appendix E.3 defines the benchmark protocol; no measured latency, mean, standard deviation or throughput is filled in here.

The demonstrated delivery route is source execution in a Windows virtual environment. No PyInstaller build log, version-matched packaged artifact, clean-machine acceptance record or offline execution record was available. Standalone executable and fully offline deployment verification remain future work, separate from successful source startup.

4.4 User Acceptance Testing Results

UAT results await collection. Appendix B provides the tasks, questionnaire and scoring rules; the tables below reserve the actual participant and outcome data. H2 and the user-performance component of H3 will be assessed after those responses are available.

4.4.1 Participant Demographics

Table 4.6. UAT participant profile — awaiting actual responses

Characteristic	Required result	Status
Recruited/consented/completed participants	[TO COMPLETE: actual counts and exclusions]	Not collected
Participant role	[TO COMPLETE: role counts, with actual denominator]	Not collected
Pathology experience	[TO COMPLETE: predefined experience groups]	Not collected
Programming/AI experience	[TO COMPLETE: experience groups]	Not collected
Prior use of pathology tools	[TO COMPLETE: actual distribution]	Not collected
Testing devices/display scales	[TO COMPLETE: actual configurations]	Not collected

Source: planned UAT protocol; no participant data have been collected.

The proposed 8–10-person target is a recruitment plan, not the sample size. Participants should be described with anonymous IDs and aggregate characteristics. Unnecessary personal data should not be collected. If the sample mainly comprises computing students rather than pathology users, that must be stated as a limitation of interpretation-focused findings.

4.4.2 Quantitative Results

Table 4.7. UAT quantitative summary — awaiting valid data

Measure	Reporting rule	Result
Valid SUS responses	All ten items in 1–5	[TO COMPLETE]
SUS mean and standard deviation	Score each participant before averaging	[TO COMPLETE]
SUS median, IQR and range	Report alongside the small-sample mean	[TO COMPLETE]
Independent task completion	Independent successes / actual attempts	[TO COMPLETE]
Assisted completion	Report separately from independent success	[TO COMPLETE]
Critical errors and assistance	Count using the predefined task rules	[TO COMPLETE]

Source: planned SUS/task analysis rules (Brooke, 1996); results await actual responses.

No comparison with the SUS benchmark is possible before valid responses exist. A future confidence interval should include the number of complete forms and assumptions, and should not be used to suggest population-level certainty from a small convenience sample. Missing-data and exclusion rules must be applied before inspecting the desired conclusion.

4.4.3 Feature Usage and Ratings

Table 4.8. Feature ratings — awaiting actual use and responses

Feature	Required usage/result evidence	Result
Single-image workflow	Actual attempts, completion and ease rating	[TO COMPLETE]
Tiled map/legend/tile details	Actual use, interpretation correctness and rating	[TO COMPLETE]
Comparison and linked navigation	Alignment task completion and rating	[TO COMPLETE]
Segmentation inspection	Use of overlay/map/legend and rating	[TO COMPLETE]
History and rerun	Recorded-mode task completion and rating	[TO COMPLETE]
Export, cancel and defaults	Completion, assistance and feature-specific rating	[TO COMPLETE]

Source: planned feature-specific UAT questions; results await actual responses.

Each feature requires its own valid-response denominator. “Not used” is a separate response and must not be converted into a dissatisfaction score. A task successfully completed with assistance should not be labelled an independent success. Interpretation answers should be compared with the prepared semantic answer key rather than agreement with an unverified tissue prediction.

4.4.4 System Performance and Acceptability

Participant-reported responsiveness should be reported separately from instrumented runtime. The questionnaire asks whether progress/cancel states were understandable and whether the application was acceptable for the stated educational/research task. It does not ask participants to certify diagnostic safety or model accuracy.

[TO COMPLETE: actual per-task timing distributions, device/CPU/RAM details, observed freezes/crashes if any, perceived responsiveness ratings and acceptance responses. Distinguish model loading, single-image inference, tiled inference, saving and export.]

Test-suite durations and the diagnostic evaluation-loop duration are excluded from inference latency claims. The Windows evaluation includes input comparison and CSV writing, and no repeated stage-level benchmark has been recorded. Runtime observations in UAT will be reported separately from the protocol in Appendix E.3.

4.4.5 Qualitative Feedback Analysis

Open-ended responses will be coded against a documented scheme, initially covering navigation, interpretation, responsiveness, accessibility, history/export and suggested improvements. New themes may be added if the responses justify them. The analysis should retain a short audit trail from anonymous response to code to theme, include contrary cases and avoid claiming saturation from a small pilot.

[TO COMPLETE: actual theme counts, anonymised quotations with consent, supporting participant IDs and the changes accepted or deferred. No participant quotation is supplied before collection.]

The engineering observations, startup record and completed Windows regressions are discussed in Section 4.3. UAT responses will be analysed under the participant and task protocol in Appendix B rather than pooled with software tests.

4.5 Discussion

4.5.1 Achievement of Research Objectives

Table 4.9. Objective achievement and outstanding evidence

Objective	Implemented outcome	Evidence-based assessment
O1: local no-code workflow	PyQt5 application, background CPU work and cancel	Windows source startup and complete 370-test suite passed; clean offline executable remains unverified.
O2: model loading/validation	Supported formats, metadata reconstruction and probe checks	Windows real-classifier loading/validation and automated loader/path cases passed; semantic safety of arbitrary models is not established.

Objective	Implemented outcome	Evidence-based assessment
O3: inference tasks	Whole-image/tiled classification and segmentation	Default-Pillow Single image core: 90.68% accuracy / 88.59% macro-F1 on 7,180 native 224 × 224 patches. Segmentation and tiled predictive evaluation pending.
O4: result inspection	Charts, overlays, legends, spatial detail and linked comparison	Native Windows chart, legend, coordinate/grid and comparison regressions passed; formal real-image inspection tasks and UAT pending.
O5: export	PNG/PDF with task-specific metadata/artifacts	Native Windows temporary-artifact/report cases passed; real-image export demonstration pending.
O6: persistent records	SQLite and saved analysis/presentation artifacts	Native Windows persistence/artifact regressions passed; immutable per-run identities and backup not implemented.
O7: retrieval/rerun	Search, local-date filter, restore and recorded rerun	Native Windows search, calendar, restore and recorded-parameter rerun cases passed; efficiency awaits UAT.

Source: the implementation, completed Windows software tests, classifier evaluation and source-startup records; manual acceptance and UAT retain separate statuses.

The implementation makes previously implicit choices visible: Single image versus Tiled analysis, source crop size versus network input, named resize backends, and tile-class coverage versus pixel coverage. The Windows patch evaluation supplies a measured default-path classifier result, and the 370-test native Windows run verifies the implemented software contracts under the documented test conditions.

H1 has supporting implementation tests, real-classifier loading and patch-level prediction evidence. Passing native Windows restoration and recorded-rerun checks support the technical component of H3. H2 and user-efficiency claims await UAT; the predictive and system-level limits are consolidated in Section 4.5.3.

4.5.2 Comparison with Existing Solutions

ZY-Path is narrower than QuPath, Slideflow and Cytomine in whole-slide, annotation, training and collaborative functionality. Its value lies in a focused local arrangement of model-aware inference, spatial presentation and persistent per-run records. WSInfer provides a clear precedent for accessible pretrained-model use; the contribution here is the particular desktop workflow and its tested state semantics, not the invention of no-code pathology inference.

No comparative trial, installation-time study or task-time benchmark against another platform was carried out. It is therefore inappropriate to claim that ZY-Path is faster, more accurate, more usable or uniquely reproducible. A future comparison should use matched tasks and intended users, document the configuration of each tool and include the capabilities that the current project omits.

4.5.3 Limitations and Constraints

Classification performance is specific to its data and preprocessing. The 90.68% accuracy and 88.59% macro-F1 are reproduced by the original evaluation and the application-default Single image core on native 224 × 224 test patches. That agreement does not establish performance after resizing arbitrary source crops or under different physical scales. Stratified patch-level splitting and disjoint file paths do not establish patient/slide independence or exclude content duplicates. Calibration and tiled-map ground truth were not evaluated.

Large raster tiling is implemented, but native WSI pyramid reading, physical-scale extraction and memory-bounded streaming remain outside scope. Full-resolution arrays and restored masks consume memory. The editable 512 × 512 classification starting value has unconfirmed training-scale equivalence, and rectangular edge crops are resized to the fixed network input.

Segmentation is a supervisor-provided model integration with a documented inference procedure. I did not train it, and its training dataset and Dice/IoU report were not obtained. Controlled tests establish software geometry and statistics rather than segmentation accuracy; a real-weight demonstration and aligned ground-truth evaluation remain necessary.

The complete 370-test native Windows suite provides automated regression evidence for the evaluated source state. Formal manual task acceptance and UAT remain incomplete, including authorised real-image screenshots, representative display configurations and real-model workflow observations. Assistive-technology testing and coverage measurement have not been completed. These activities address usability and deployment conditions beyond the controlled software cases.

Runtime and deployment require further measurement. The documented Windows execution is CPU-based; its installed RTX 4060 was not used. Repeated cold/warm stage timings, memory measurements, PyInstaller build records, clean-machine acceptance and offline dependency checks are unavailable, so the current delivery level is source execution.

Model compatibility and provenance are constrained. Structural checks cannot guarantee semantic labels, calibrated confidence or safety of arbitrary serialised modules. Records reference mutable paths and do not provide immutable per-run identities, automatic backups, multi-user access control or an atomic transaction over external artifacts. These limits restrict the system to trusted-model educational and exploratory research use.

4.5.4 Implications for Future Development

The immediate priorities are to complete formal real-image Windows acceptance, collect UAT responses and retain the passing regression suite as the baseline for further changes. The default application patch evaluation is complete; further model work should examine patient/slide grouping, different source dimensions and physical scale. Segmentation evaluation requires authorised aligned masks.

Stronger provenance should then add immutable file identifiers, software/dependency manifests and versioned analysis schemas. A transactional artifact manifest and backup/recovery workflow would improve persistence robustness. Verified packaging should include model/configuration licensing, required local caches and a clean Windows test. Native accessibility/contrast checks and screen-reader feedback should inform further UI refinement.

Only after these foundations should the project consider physical-scale-aware WSI reading, memory-bounded streaming, acceleration or explainability. Each extension

changes the validation burden. A classifier heatmap must remain distinguishable from segmentation, and any explanation view must remain distinguishable from measured model validity.

4.6 Summary

ZY-Path implements model-aware inference, spatial inspection and record continuity. The Windows default-Pillow Single image core reproduces 90.68% accuracy and 88.59% macro-F1 on 7,180 labelled patches. The final native Windows full suite completed with 370 passed, 0 failed, 0 errors and 0 skipped; the Linux installation retained one real-segmentation-file availability failure. UAT and the model, performance and deployment limitations remain as specified in Section 4.5.3.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

This capstone has developed ZY-Path from a CP1 proposal into a source-based desktop application for educational and exploratory histopathology image analysis. It combines compatible pretrained-model loading, CPU inference, graphical result inspection, local history and export through a PyQt5/PyTorch/SQLite implementation. The original objective of reducing the need to write inference code is addressed through a constrained graphical workflow rather than unrestricted model compatibility.

The implementation adds explicit single-image and tiled classification, separate segmentation semantics, linked source/result inspection, task-specific model defaults and cooperative cancellation. In particular, tiled analysis retains actual crop coordinates and areas, exposes its sampling settings and labels coverage as an estimate. History and rerun preserve the recorded task/model/mode instead of silently applying current preferences. These details connect interface behaviour to the meaning of the numerical result.

The Windows application-default Pillow evaluation produced 6,511 correct predictions from 7,180 native-sized test patches, or 90.68% accuracy and 88.59% macro-F1. It matches the original OpenCV/Albumentations evaluation on every predicted label, while probabilities show small numerical differences. Separately, all 370 native Windows software tests passed without errors or skips, supporting the implemented loading, inspection, persistence, rerun, cancellation and export contracts. Formal manual acceptance and predictive generalisation retain the limits in Section 4.5.3.

The research questions are answered at different levels. A model-aware local GUI workflow is implemented, and recorded restoration/rerun logic supports continuity under stable files and dependencies. Classification has a documented test result; perceived usability, interpretation and efficiency remain questions for the planned UAT. The contribution is an integrated research/education workflow whose acceptance and deployment boundaries are specified in Section 4.5.3.

5.2 Future Work and Recommendations

Further evaluation should examine patient/slide-separated classifier performance, source-scale sensitivity, resize behaviour beyond native-sized patches and segmentation against authorised masks. Formal real-image Windows acceptance should complement the completed automated suite, which provides the regression baseline for subsequent development.

The second priority is user evaluation. Run the planned task-based pilot with intended users, score valid SUS responses correctly, separate assisted from independent completion and analyse interpretation errors as well as preferences. Revise the interface in response to actual findings, then rerun the relevant regressions and report which changes improved observed tasks. A later controlled comparison could investigate workflow advantages against an established platform without assuming the result in advance.

The third priority is reproducibility and deployment. Add file checksums, dependency manifests and versioned metadata to saved records; define artifact transaction/recovery behaviour and backups; package the application with all permitted resources; and test it on a Windows machine without the development environment. Security and model-licensing review should accompany distribution, especially where serialized model objects remain supported.

Longer-term work may include memory-bounded WSI reading with physical-scale metadata, optional acceleration, calibration/uncertainty analysis and carefully validated explanation views. Such work would expand the project's scope and require new requirements, datasets and tests. It should not be described as an implemented outcome of this version.

5.3 Research Contributions

The first contribution is a local classification workflow connecting supported PyTorch inference to a graphical interface and persistent analysis records. A separately integrated segmentation pathway broadens inspection options, with its evidence and evaluation limits explicitly distinguished from the trained classifier.

The second contribution is a practical separation of inference semantics in the interface and stored metadata. Whole-image classification, tile-class coverage and

segmentation pixel coverage remain distinguishable. Source-crop geometry, edge areas, resize choices, class order and the unconfirmed training scale are exposed rather than hidden behind a visually plausible output.

The third contribution is a tested approach to inspection and state continuity: shared coordinate transforms, linked views, vector tile boundaries, responsive layouts, recorded rerun settings and safe cancellation preserve meaning across user actions. The regression cases document defects that purely static or offscreen screenshots had not been sufficient to assess.

The fourth contribution is an evaluation package linking requirements, named software tests, the classification training/test outputs and platform-specific execution records. It includes class-wise metrics, a confusion matrix, the completed native Windows suite and prepared UAT instruments. The work contributes an engineering integration and its documented evaluation rather than a new neural architecture.

5.4 Summary

ZY-Path implements the central CP1 workflow and extends it with explicit spatial analysis and stronger state management. The measured classifier result and the completed 370-test native Windows suite complement source-startup observations. UAT, formal manual acceptance, segmentation evaluation and verified packaging are the next steps toward stronger usability, generalisation and deployment claims.

References

Aiforia (n.d.) Aiforia Create. Product information. https://www.aiforia.com/aiforia-create

Bangor, A., Kortum, P. and Miller, J. (2009) ‘Determining what individual SUS scores mean: Adding an adjective rating scale’, Journal of Usability Studies, 4(3), pp. 114–123.

Bankhead, P. et al. (2017) ‘QuPath: Open source software for digital pathology image analysis’, Scientific Reports, 7, article 16878. https://doi.org/10.1038/s41598-017-17204-5

Brooke, J. (1996) ‘SUS: A “quick and dirty” usability scale’, in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) Usability Evaluation in Industry. London: Taylor & Francis, pp. 189–194.

Dolezal, J.M., Kochanny, S., Dyer, E., Ramesh, S., Srisuwananukorn, A., Sacco, M., Howard, F.M., Li, A., Mohan, P. and Pearson, A.T. (2024) ‘Slideflow: Deep learning for digital histopathology with real-time whole-slide visualization’, BMC Bioinformatics, 25, article 134. https://doi.org/10.1186/s12859-024-05758-x

fpdf2 Contributors (n.d.) fpdf2: Simple PDF generation for Python. Project documentation. https://github.com/py-pdf/fpdf2

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017) ‘On calibration of modern neural networks’, Proceedings of the 34th International Conference on Machine Learning, Proceedings of Machine Learning Research, 70, pp. 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

Huang, G., Liu, Z., van der Maaten, L. and Weinberger, K.Q. (2017) ‘Densely connected convolutional networks’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2261–2269. https://arxiv.org/abs/1608.06993

Hunter, J.D. (2007) ‘Matplotlib: A 2D graphics environment’, Computing in Science & Engineering, 9(3), pp. 90–95. https://doi.org/10.1109/MCSE.2007.55

Kaczmarzyk, J.R., O’Callaghan, A., Inglis, F., Gat, S., Kurc, T., Gupta, R., Bremer, E., Bankhead, P. and Saltz, J.H. (2024) ‘Open and reusable deep learning for pathology with WSInfer and QuPath’, npj Precision Oncology, 8, article 9. https://doi.org/10.1038/s41698-024-00499-9

Kather, J.N. et al. (2019) ‘Predicting survival from colorectal cancer histology slides using deep learning: A retrospective multicenter study’, PLOS Medicine, 16(1), e1002730. https://doi.org/10.1371/journal.pmed.1002730

Kather, J.N. (n.d.) deep-stroma-histology: Code and dataset guidance for histological tissue classification. Project repository. https://github.com/jnkather/deep-stroma-histology

Kreibich, J.A. (2010) Using SQLite. Sebastopol, CA: O'Reilly Media.

Litjens, G. et al. (2017) 'A survey on deep learning in medical image analysis', Medical Image Analysis, 42, pp. 60–88. https://doi.org/10.1016/j.media.2017.07.005

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S. and Guo, B. (2021) 'Swin Transformer: Hierarchical vision transformer using shifted windows', Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 10012–10022. https://arxiv.org/abs/2103.14030

Marée, R. et al. (2016) 'Collaborative analysis of multi-gigapixel imaging data using Cytomine', Bioinformatics, 32(9), pp. 1395–1401. https://doi.org/10.1093/bioinformatics/btw013

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 152–158. https://doi.org/10.1145/191666.191729

NumPy Developers (n.d.) NumPy user guide. https://numpy.org/doc/stable/user/

Paszke, A. et al. (2019) 'PyTorch: An imperative style, high-performance deep learning library', Advances in Neural Information Processing Systems, 32. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pillow Contributors (n.d.) Pillow documentation: Image processing and resampling. https://pillow.readthedocs.io/en/stable/

PyInstaller Development Team (n.d.) PyInstaller documentation. https://pyinstaller.org/en/stable/

PyTorch Contributors (n.d.) torch.load; inference_mode; serialization semantics. PyTorch documentation. https://pytorch.org/docs/stable/

Riverbank Computing (n.d.) PyQt5 reference guide. https://www.riverbankcomputing.com/static/Docs/PyQt5/

Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional networks for biomedical image segmentation', in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, pp. 234–241. https://doi.org/10.1007/978-3-319-24574-4_28

SQLite (n.d.) Zero-configuration. SQLite documentation. https://sqlite.org/zeroconf.html

Tan, M. and Le, Q.V. (2019) 'EfficientNet: Rethinking model scaling for convolutional neural networks', Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research, 97, pp. 6105–6114. https://proceedings.mlr.press/v97/tan19a.html

The Qt Company (n.d.) Qt 5 documentation: QThread, QScreen, QGraphicsView and high-DPI support. https://doc.qt.io/qt-5/

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 5 October 2023. https://www.w3.org/TR/WCAG22/

Xiao, T., Liu, Y., Zhou, B., Jiang, Y. and Sun, J. (2018) ‘Unified perceptual parsing for scene understanding’, Proceedings of the European Conference on Computer Vision, pp. 418–434. https://arxiv.org/abs/1807.10221

Yen, P.-Y. and Bakken, S. (2012) ‘Review of health information technology usability study methodologies’, Journal of the American Medical Informatics Association, 19(3), pp. 413–422. https://doi.org/10.1136/amiajnl-2010-000020

Appendices

Appendix A: Project Timeline and Planning

A.1 Proposal baseline

Tables A.1 and A.2 retain the CP1 planning baseline as a stored record. References there to ResNet-50, ImageNet preprocessing, a >94% target, coverage targets and executable packaging describe the proposal, not the final implementation or achieved results. Table A.3 gives the updated status; Chapters 3 and 4 report the implemented DenseNet/raw255 contract and measured outcomes.

Table A.1. Original CP1 planning baseline, retained from the submitted proposal

CP1 Week	Phase Theme	Key Activities	Deliverables
3	Project Initiation	Finalize project title; articulate the no-code core concept; complete and submit the Student Assessment Form (SAF) for supervisor approval.	Approved SAF with registered title
4	Problem Identification	Identify three core gaps in existing platforms; draft the initial system flowchart; analyse QuPath, Slideflow and WSInfer; tabulate the feature comparison.	Draft problem statement and initial flowchart
5	Background Research	Collect and summaries ≥10 peer-reviewed articles covering digital pathology, deep learning, no-code AI and clinical usability; draft Chapter 1.1–1.2.	Annotated bibliography (≥10 articles)
6	Objective Definition	Define three research questions and seven specific objectives; delineate in-scope and out-of-scope items; articulate significance and SDG 3/4 alignment.	Completed Chapter 1

CP1 Week	Phase Theme	Key Activities	Deliverables
7	Literature Extension	Add five further articles (total ≥15); focus on PyTorch deployment, SQLite management and HCI heuristics; refine flowcharts into Figures 3.1 and 3.2; draft the three-layer architecture.	Expanded bibliography and detailed design diagrams
8	Semester Break	Cognitive digestion of collected literature; informal reading; mental preparation for the intensive writing phase.	None
9	Chapter 2 Completion	Synthesize all reviewed literature into coherent Chapter 2 sections; finalize and justify the development toolset (Python 3.10, PyQt5, PyTorch, SQLite, Matplotlib, fpdf2 and PyInstaller).	Completed Chapter 2 and technology justification table
10	Methodology Documentation	Document Chapter 3, including architecture, class diagram, model constraints, preprocessing, database schema and use cases UC1–UC4; formalize testing and evaluation methodology; prepare the initial Gantt chart.	Completed Chapter 3 and draft CP1 work plan
11	CP2 Planning	Draft a detailed CP2 work plan defining coding sprints, testing phases, evaluation schedules and thesis milestones; draft the abstract (≤300 words).	Draft CP2 work plan and abstract draft
12	Plan Finalization	Refine the CP2 plan based on Chapter 3 complexity; add explicit testing and debugging buffers; finalize the abstract incorporating supervisor feedback.	Finalized CP2 work plan and finalized abstract

CP1 Week	Phase Theme	Key Activities	Deliverables
13	Integration	Merge CP1 and CP2 plans into a unified Gantt chart; cross-check deliverables against research questions; standardize Harvard citations; verify figure/table numbering and heading hierarchy.	Fully formatted draft document
14	CP1 Submission	Compile Chapters 1–4, abstract and all appendices into a single PDF; submit via eLearn before the deadline; corresponds to Go/No-Go Gate A.	CP1 submission (Gate A passed)

Source: CP1 proposal planning baseline, retained as proposal-stage planned activities.

Table A.2. Original CP2 planning baseline, retained from the submitted proposal

CP2 Week	Development Layer	Key Activities	Milestone
1–2	Core Architecture	Configure the Windows 10/11 environment (Python 3.10, VS Code, Git/GitHub and virtual environment); implement the ui/, core/ and data/ package structure; develop ModelLoader with probe-tensor validation and Preprocessor with a standard torchvision pipeline (Resize 224×224, ToTensor, ImageNet normalisation and unsqueeze).	Functional project skeleton with passing unit tests

CP2 Week	Development Layer	Key Activities	Milestone
3–4	Presentation Layer	Build the PyQt5 GUI: MainWindow with QSplitter (three regions); InferencePanel (built-in toggle, QFileDialog, upload thumbnail, Classification/Segmentation radios and Run Inference); ResultPanel (classification bar chart and segmentation overlay); HistoryPanel (QTableWidget, search and date filters); wire the signal-slot mechanism.	Fully navigable GUI shell with all panels interactive
5–6	Inference Layer	Develop InferenceEngine (QThread subclass) to execute the background inference pipeline: ModelLoader.load() → Preprocessor.transform() → torch.no_grad() forward pass → OutputParser.parse_cls()/parse_seg(); integrate the built-in ResNet-50 fine-tuned on NCT-CRC-HE-100K (nine-class colorectal tissue dataset), with automatic loading on startup.	Functional end-to-end inference pipeline
6–7	Data Layer	Implement DatabaseManager (sqlite3) to create the inference_records table with complete metadata; develop parameterised LIKE search and date-range filtering with defensive try/except handling; build ExportManager with export_png() (Matplotlib, 150 DPI) and export_pdf() (fpdf2 A4 report).	Persistent history management and professional report generation

CP2 Week	Development Layer	Key Activities	Milestone
7–8	Integration & Refinement	Validate the four core use cases (UC1–UC4) end-to-end; stress-test signal-slot communication across layers; refine the UI according to Section 3.4, including the clinical color palette, WCAG 2.2 Level AA contrast, tooltips and progress indicators.	Stable, accessible application ready for formal testing (Gate B)
9–10	Testing & Validation	Develop a comprehensive pytest suite covering ModelLoader, Preprocessor, OutputParser, DatabaseManager and ExportManager, targeting ≥80% coverage; validate the built-in ResNet-50 on an NCT-CRC-HE-100K test subset (>94% accuracy); test ≥3 custom PyTorch architectures for compatibility.	Comprehensive test report with validated accuracy benchmarks
11	Usability Evaluation	Recruit 8–10 participants, including pathology students, biomedical researchers and clinical technologists; conduct heuristic evaluation based on Nielsen's ten heuristics, a SUS questionnaire (0–100 scale) and semi-structured think-aloud interviews.	Completed SUS scores and qualitative interview transcripts

CP2 Week	Development Layer	Key Activities	Milestone
12	Results & Packaging	Analyse SUS scores against the benchmark of 68; perform thematic analysis; verify UC4 reproducibility; package the application with PyInstaller (--onefile --windowed); test on clean Windows 10/11 virtual machines without Python installed.	Validated evaluation report and deployable executable (Gate C)
13	Thesis Completion	Write Chapter 4, covering implementation outcomes, SUS results, accuracy benchmarks and gap validation; write Chapter 5, covering the conclusion, answers to the three RQs, contributions, limitations and future work, including WSI support, model training, multi-user support, XAI, cross-platform deployment and a model repository.	Complete thesis draft
14	Final Submission	Proofread and format the complete thesis; compile the final PDF; prepare the executable and supplementary materials; submit all deliverables via eLearn.	Final submission concluding the project lifecycle

Source: CP1 proposal's CP2 plan; proposed model, performance and deployment targets are not achieved-result statements.

A.2 Implementation and outstanding milestones

Table A.3. Evidence-based milestone status

Milestone	Available evidence	Current status
Core interface/inference/persistence integration	Project modules and completed 370-test native Windows run	Implemented under supported contracts

Milestone	Available evidence	Current status
Linked original/result views	Source history including revision 206b51f and current tests	Implemented; final native capture pending
Cancellation and independent model defaults	Revision 12633fb and lifecycle/default tests	Implemented with cooperative-stop limits
Settings/history semantics	Revisions 643099b and 562ce39	Implemented and regression-tested
Settings guide and responsive comparison refinement	Revision e058354	Implemented and tested with fixtures
Full-area result toggles and English calendar	Revision 1f3e01e	Baseline plus lifecycle/layout repairs; evaluated source identified in Appendix C
Classifier training and evaluation	30-epoch record, epoch-26 checkpoint, 7,180 predictions and matching metrics	Default-Pillow patch evaluation completed; 90.68% accuracy / 88.59% macro-F1; generalisation limits retained
Formal UAT and Windows acceptance record	Windows automated regressions complete; source startup recorded; no UAT responses or completed manual task sheets	Automated Windows verification complete; formal manual acceptance and UAT pending
Clean-machine packaged distribution	No verified executable/specification supplied	Pending
Native Windows full-suite regression	Final log/XML/exit code; 370 distinct passing cases; source identity recorded	Completed on 19 Oct 2026; 0 failures, errors or skips

Source: project revision history, classification experiment outputs and Windows execution record.

A.3 Completion sequence

Before submission, complete authorised GUI screenshots, formal manual acceptance, UAT and the remaining cover fields. Retain the completed 370-test Windows result and the default-Pillow patch evaluation with their actual source and dataset scope. Final formatting and clean-machine packaging claims follow their corresponding review and execution evidence.

Appendix B: User Acceptance Testing Details

B.1 Participant information and consent briefing

Study purpose: evaluate the usability of ZY-Path for educational/research image-analysis tasks. The study does not assess a participant's clinical competence or establish model diagnostic accuracy. Participation is voluntary. Participants may stop at any time or decline an optional question. Any recording requires separate permission; no identifiable patient image should be used.

Before starting, confirm institutional requirements and any required approval. Allocate an anonymous participant ID. Record the application revision, device/display scale and prepared sample/model identifiers. Explain that some predictions may be incorrect and that the participant should evaluate the workflow rather than infer a clinical diagnosis. Agree how anonymised responses and any permitted quotations will be stored and used.

Consent fields: participant ID [blank]; consent to participate [yes/no]; consent to anonymised quotation [yes/no]; consent to screen/audio recording [yes/no/not requested]; date [blank]; researcher initials [blank]. Do not collect a name where an anonymous ID is sufficient. A participant who declines should not be entered as a completed response.

B.2 Background questions

Record role (e.g., biomedical/pathology student, researcher, educator, other); pathology-image experience (none, occasional, regular); programming/AI experience (none, basic, intermediate/advanced); previous pathology-tool use (yes/no; optional tool name); and testing device/display scale. Age grouping is optional and should be collected only if needed for the approved study. Do not imply that all recruited users are qualified pathologists.

B.3 Standard task script and observation rules

Table B.1. UAT tasks and success criteria

Task	Participant instruction	Observer success criterion
T01	Load the supplied image and run Single image with the approved model	Correct image/model/mode and completed output identified without coding

Task	Participant instruction	Observer success criterion
T02	Select Tiled analysis, set the specified source width/height and run	Correct explicit mode and source dimensions are used
T03	Find a tile's class/confidence and explain coverage	Identifies actual tile detail; states coverage is a crop-class area estimate, not pixel ground truth
T04	Compare a selected region with the original and use Fit/zoom	Same source region remains aligned; participant can return to Fit
T05	Inspect the supplied segmentation result	Distinguishes segmentation labels and pixel coverage from classification tiles
T06	Search/date-filter history and restore the nominated record	Correct stored record shown without initiating inference
T07	Change a current preference, then rerun the stored record	Recorded model/mode/parameters used; new record distinguished from restored one
T08	Export the nominated completed result	Correct PNG/PDF and actual task/model metadata identified
T09	Start the supplied workload and cancel it	Recognises request/acknowledgement; no partial completed record saved
T10	Save a supplied validated default and restore the built-in default	Correct task's preference changes; other task/history remain unaffected

Source: author’s UAT task protocol.

For each attempted task record start/end or elapsed seconds, outcome (independent success, assisted success, unsuccessful, not attempted), assistance count, critical error and a brief observation. Define a stop rule consistently before testing. An intended cancellation is successful when safe acknowledgement and record preservation occur, not when the model stops instantly. If a prepared file is unavailable, record an environment-blocked attempt rather than blaming the participant or silently substituting another task.

The interpretation answer key is semantic: source tiles are resized to network input; confidence is a model score; tile coverage uses actual crop areas; segmentation has independent labels; and stored rerun uses recorded settings. It must not require agreement with an unverified predicted tissue category.

B.4 System Usability Scale questionnaire

After completing the tasks, ask participants to rate the following standard SUS statements from 1 (Strongly disagree) to 5 (Strongly agree). Keep their order and wording consistent; analyse these ten items separately from the project-specific questions (Brooke, 1996).

8. I think that I would like to use this system frequently.
9. I found the system unnecessarily complex.
10. I thought the system was easy to use.
11. I think that I would need the support of a technical person to be able to use this system.
12. I found the various functions in this system were well integrated.
13. I thought there was too much inconsistency in this system.
14. I would imagine that most people would learn to use this system very quickly.
15. I found the system very cumbersome to use.
16. I felt very confident using the system.
17. I needed to learn a lot of things before I could get going with this system.

B.5 Feature-specific ratings and open questions

Rate each feature used from 1 (Very difficult/unclear) to 5 (Very easy/clear), with a separate Not used option: model/image selection; mode/tile settings; legend/statistics; tile details; original comparison/linked views; segmentation inspection; history/rerun; export; cancellation/progress; and settings/help. Report denominators for each feature. These ratings do not modify the SUS score.

Ask: Which part was easiest? Which part was confusing? Was any statistic or colour meaning unclear? Could you recover from an error or cancel safely? What would you change first? Would you use this system for the stated educational/research task (yes, maybe, no), and why? Do not ask a leading question that assumes the application is better than another system.

B.6 Scoring, missing data and reporting

The accompanying workbook contains blank participant, task, SUS and feature-rating sheets. SUS scoring requires ten numeric integers in 1–5 and applies Equation 3.4. A blank/incomplete form remains unscored. In task results, independent completion and assistance are separate outcomes; feature “Not used”

is excluded from numerical ratings. Store anonymous identifiers consistently across sheets, document exclusions and retain the unmodified raw responses separately.

For the pilot, report actual N, recruitment method, device conditions, SUS distribution, completion denominators, assistance/errors and qualitative themes. If the study lacks a comparator, describe perceived usability rather than claiming causal improvements in efficiency. Add de-identified quotations only with consent and never fill empty Chapter 4 tables using generated participants.

Appendix C: Implementation and Evidence Index

C.1 Source baseline

Source baseline: revision 1f3e01e7866d1b1f78103d817214f7dbb1cc5a56, supplemented by the lifecycle and responsive-layout repairs evaluated on 19 October 2026. The passing Windows source state includes changes in nine production modules and four test modules. The evidence package preserves the tracked difference, the thirteen per-file checksums and the reviewed source supplement, including the new worker-lifetime test file; no new software commit is implied. Table C.1 identifies each execution record.

Table C.1. Principal evidence sources

Evidence	Source	Supports
E01	core/model_loader.py	Supported representations, size limit, metadata/probe checks, UPerNet reconstruction
E02	core/preprocessor.py; supervisor-provided train_crc_dn21.py and experiment records	RGB/raw255/input size, selectable resize, reference transforms
E03	core/classification_tiles.py	Crop geometry, edge areas, nine-class palette, counts and aggregate probabilities
E04	core/large_image_segmentation.py; core/inference_engine.py; supervisor-provided inference notebook	Independent segmentation defaults, fusion/restoration and worker cancellation
E05	ui/main_window.py; data/database_manager.py	Artifact/record creation, recorded rerun, local data paths

Evidence	Source	Supports
E06	ui/classification_result.py; ui/segmentation_result.py; ui/image_comparison.py; ui/zoomable_image_view.py	Spatial views, legends, linked coordinates and vector grid
E07	ui/settings_dialog.py; ui/model_settings.py; ui/guide_view.py	Preferences, per-task defaults and reading-aware help
E08	ui/result_check_box.py; ui/theme.py; ui/history_panel.py	Full-area toggle input, focus styling and English calendar
E09	data/export_manager.py; tests/test_main_window_export.py	PNG/PDF and recorded task/model metadata
E10	Dated Linux and Windows collection, runtime, subset and complete-suite records	Environment/date-specific outcomes, commands and prerequisite states
E11	Classification manifests, config.json, metrics.csv, training logs and best_model.pth	Recorded data split, 30 epochs, selected epoch and checkpoint identity
E12	Classification test-7k predictions/report/matrix/summary files	Recorded 7,180-image metrics; independently recomputed CSV consistency
E13	Supervisor-provided predict-upernet-tme-tissue-overlap.ipynb	Segmentation architecture, independent preprocessing, labels and overlap fusion
E14	19 Oct 2026 originals: pytest-full.log, pytest.xml, pytest-exit-code.txt, commit.txt, test-date.txt and environment files	First Windows invocation: collection error, exit 2; resolved by restoring the original training script
E15	19 Oct 2026 original windows-app.log	Successful startup/real-classifier validation and limited window states; no completed image inference or all-feature acceptance
E16	19 Oct Windows rerun: live progress, exit code and model-availability records	Earlier 359-item invocation stopped with no final totals; failure paths subsequently repaired and exercised in the final suite
E17	19 Oct default-Pillow predictions, class report, matrix, summary and environment	7,180 completed patch predictions, model identity, actual preprocessing, reference label agreement and probability differences
E18	19 Oct, 22:24 Windows related log/XML, exit code and Qt snapshot	131 passed; native windows platform; independent subset, not added to the full-suite total
E19	19 Oct, 22:26 Windows full log/XML, exit code, Qt snapshot and source hashes	370 passed; 0 failed/errors/skipped; 15 warnings; 194.10s; evaluated baseline plus repairs
E20	Final 19 Oct Linux full log/XML and source supplement	369 passed, 1 missing-real-segmentation-file failure, 0 collection errors; no skipped tests

Source: project modules, experiment files and identified test records listed in the table.

C.2 Evidence interpretation

The evidence archive separates the final completed invocations from earlier collection errors, runtime termination and diagnostic subsets. Windows completed 370 tests with no failures, errors or skips; its preceding 131-test subset is not added to that total. Linux completed 369 passing tests and one missing-segmentation-file availability failure. The default-Pillow patch evaluation remains the source of classification metrics, independently of these software-test counts.

Appendix D: Manual Windows Acceptance and Screenshot Checklist

D.1 Native acceptance record

For each manual acceptance case record date, tester, source state, Windows/Python/dependency versions, hardware, Qt display metrics, model/input identity, expected outcome, actual outcome and evidence file. The source-startup log gives partial manual observations, while the completed Windows suite supplies automated coverage for several corresponding behaviours. Table D.1 remains a manual acceptance checklist: its outcomes require the specified real-image task records and must not be filled from automated test totals alone.

Table D.1. Final Windows acceptance cases

Case	Action	Expected evidence
W01	Launch and restore previous geometry	Normal window fits current available screen and retains usable actions
W02	Maximise, restore, shrink, maximise, restore	Native states and normal geometry behave correctly without forced fullscreen
W03	Dock narrow, inspect comparison, restore wide	Vertical/horizontal adaptation works without Reset; views remain aligned
W04	Fit, 1:1, zoom and resize a 49-tile result	Complete aligned 7 × 7 grid; 480/464-pixel edge dimensions recorded
W05	Click toggle labels, padding and indicators; use Tab/Space	One transition per click; mouse/keyboard focus styles appropriate

Case	Action	Expected evidence
W06	Open both history calendars	English month/weekdays; correct selected dates and filtering
W07	Restore/rerun after changing current defaults	Recorded model/mode/parameters retained; restore itself does not rerun
W08	Cancel a real workload and close while busy	Safe acknowledgement/cleanup; no partial completed record
W09	Scroll/maximise Settings at common display scales	All content reachable, no artificial trailing height or clipped controls
W10	Export, reopen PDF/PNG and inspect companion files	Correct task/model/statistics; files readable; no unwanted identifying data
W11	Validate/save/restore defaults per task	Clear invalid/missing-file handling and task independence
W12	Move required files and attempt restoration/rerun	Actionable missing-file errors; no silent model substitution

Source: author's Windows acceptance protocol. Expected evidence describes the acceptance criterion, not an observed Pass.

D.2 Required figure replacements

Figures 1.1 and 4.1, 4.3–4.6 and 4.9 explicitly reserve the required evidence. The accompanying Chinese capture guide gives step-by-step framing and filenames. Figures 4.2, 4.7 and 4.8 are labelled development captures; retain that label or replace them with equivalent current Windows captures and update the caption's environment/source description. Screenshot dates, model identity and software revision must correspond to the actual captured run.

Appendix E: Independent Model Evaluation Protocol

E.1 Classification

The completed classification evaluation used CRC-VAL-HE-7K with 7,180 labelled images. The selected epoch-26 checkpoint SHA-256 is 8f89bd825641fba276066b54432b5d73a62b11ed6c0d70bb94c9f0bc7e2fe325; the experiment file best_model.pth was renamed crc_dn21_seed42_best.pth for application use. The hash of the experiment checkpoint agrees with its evaluation

summary. The original dataset URL is https://zenodo.org/records/1214456; the manifests identify NCT-CRC-HE-100K for training/validation and CRC-VAL-HE-7K for testing.

The original test used OpenCV RGB decoding followed by Albumentations 1.3.1 Resize(224, 224), ToTensorV2 and float32 raw255, with OpenCV 4.8.1. Its environment lists Python 3.10.22, PyTorch 2.1.2+cu121, torchvision 0.16.2+cu121 and scikit-learn 1.3.2. The Windows rerun used the application's default Pillow pathway on CPU: Python 3.10.22, PyTorch 2.1.2+cpu, Pillow 10.4.0, NumPy 1.26.4 and OpenCV 4.11.0 for the input diagnostic. Both use the same nine-class order and checkpoint.

The Windows summary records Single image, pillow_bilinear, RGB float32 raw255, input size 224 × 224 and 7,180 native 224 × 224 source images. Per-image CSVs independently reproduce 6,511 correct labels, the class-wise report and the confusion counts. All reference labels agree; the largest absolute probability difference is 0.002731651. This run does not exercise resizing of larger crops. Future studies should document patient/slide grouping, duplicate checks and physical scale, and define separate ground truth for Tiled analysis.

E.2 Segmentation

The supervisor-provided inference notebook establishes the model configuration and label mapping but contains no independently annotated test-set evaluation. The segmentation training dataset, training procedure and Dice/IoU report were not obtained. A future evaluation requires authorised aligned masks, patient/slide separation, the same label vocabulary, an explicit ignored-label policy and recorded tile/stride/input settings; per-class and aggregate IoU/Dice should then be calculated with documented denominator rules.

E.3 Runtime and deployment

Use the documented Windows CPU environment as a benchmark baseline, fixing source dimensions, model, classification mode, resize backend and tile settings. Time cold model loading separately from warm repeated runs; measure preprocessing, forward inference, reconstruction/statistics, GUI display, saving and export as distinct stages. Report repeat count, mean, sample standard deviation,

median and range, with memory measurements where available. The recorded pytest durations are excluded from inference timing. Packaging verification additionally requires a matched PyInstaller build record and artifact revision, clean-machine task checks and an offline dependency test; these remain planned activities.